WYŻSZA SZKOŁA
INFORMATYKI i ZARZĄDZANIA
z siedzibą w Rzeszowie

KOLEGIUM INFORMATYKI STOSOWANEJ

Kierunek: **INFORMATYKA**
Specjalność: **Programowanie**

Patryk Rzegocki
Nr albumu studenta: 69840

Godmist – projekt fabularnej gry RPG typu dungeon crawler z turowym systemem walki

Prowadząca: mgr inż. Ewa Żesławska

INŻYNIERIA OPROGRAMOWANIA

Rzeszów 2026

Spis treści

1 Wprowadzenie, analiza problemu i wymagania 3
1.1 Cel i zakres pracy . . . 3
1.2 Analiza problemu . . . 3
1.3 Przegląd istniejących rozwiązań . . . 4
1.3.1 Baldur's Gate 3 . . . 4
1.3.2 Darkest Dungeon . . . 5
1.3.3 Clair Obscur: Expedition 33 . . . 5
1.3.4 Wnioski z analizy porównawczej . . . 6
1.4 Opis biznesowy systemu . . . 6
1.4.1 Wymagania funkcjonalne . . . 6
1.4.2 Wymagania niefunkcjonalne . . . 9
1.5 Podsumowanie rozdziału . . . 10

2 Projekt systemu i struktura rozwiązania 11
2.1 Modelowanie systemu . . . 11
2.1.1 Diagramy przypadków użycia . . . 11
2.1.2 Scenariusze przypadków użycia . . . 15
2.1.3 Diagramy aktywności . . . 18
2.1.4 Diagramy sekwencji . . . 20
2.1.5 Diagramy stanów . . . 22
2.2 Technologie i wymagania sprzętowe . . . 23
2.2.1 Technologie . . . 24
2.2.2 Wymagania sprzętowe . . . 24
2.3 Struktura systemu . . . 25
2.3.1 Opis modelu danych i trwałości . . . 25
2.3.2 Diagram ERD . . . 27
2.3.3 Diagram klas . . . 28
2.3.4 Architektura warstwowa systemu . . . 29
2.4 Podsumowanie rozdziału . . . 30

3 Interfejs użytkownika, testy i podsumowanie 31
3.1 Projekt interfejsu użytkownika . . . 31
3.2 Propozycja testów . . . 34
3.2.1 Testy funkcjonalne . . . 34
3.2.2 Testy integracyjne . . . 36
3.2.3 Testy zapisu i odczytu . . . 36
3.2.4 Testy wydajnościowe i niezawodnościowe . . . 37
3.3 Podsumowanie rozdziału . . . 38

4 Podsumowanie **39**
4.1 Ocena realizacji założeń 39
4.2 Wnioski końcowe 39
4.3 Możliwe kierunki dalszego rozwoju 39

Spis rysunków **40**

Spis tabel **41**

Spis listingów **42**

Bibliografia **43**

1. Wprowadzenie, analiza problemu i wymagania

Przedmiotem niniejszej pracy jest projekt systemu gry komputerowej *Godmist*. Jest to gra typu single player RPG, łącząca eksplorację lochów, rozwój drużyny postaci, walkę turową na siatce kwadratowej, system przedmiotów i craftingu, interakcje z postaciami niezależnymi oraz realizację misji. Projekt został przygotowany z perspektywy inżynierii oprogramowania, dlatego oprócz warstwy rozgrywki uwzględniono również strukturę danych, architekturę systemu, trwałość stanu gry oraz sposób modelowania najważniejszych procesów.

Tworzenie gry tego typu stanowi interesujące wyzwanie projektowe, ponieważ wymaga połączenia wielu współzależnych mechanizmów w jedną spójną całość. System musi jednocześnie obsługiwać eksplorację miasta i lochów, walkę wielu jednostek, rozwój postaci, zarządzanie ekwipunkiem, crafting, handel, questy oraz zapis i odczyt stanu gry. Każdy z tych obszarów posiada własne reguły, lecz dopiero ich integracja decyduje o jakości końcowego rozwiązania. Z tego powodu projekt *Godmist* może być traktowany nie tylko jako koncepcja gry, ale również jako przykład złożonego systemu informatycznego o wielu warstwach odpowiedzialności.

1.1. Cel i zakres pracy

Celem pracy jest opracowanie projektu systemu gry RPG, który umożliwia prowadzenie rozgrywki opartej na eksploracji proceduralnych lochów, rozwoju drużyny oraz zarządzaniu zasobami gracza. System ma zapewniać spójną obsługę kluczowych elementów rozgrywki, takich jak walka turowa, postęp postaci, przedmioty, NPC, misje oraz trwałość danych. Istotnym celem jest również zaprojektowanie rozwiązania w sposób modularny, tak aby możliwe było jego dalsze rozwijanie i rozbudowa o kolejne mechaniki.

Zakres pracy obejmuje analizę problemu projektowego, przegląd wybranych istniejących rozwiązań, sformułowanie wymagań funkcjonalnych i niefunkcjonalnych, przygotowanie modeli UML, opracowanie logicznego modelu danych, przedstawienie technologii i wymagań sprzętowych, zaprojektowanie interfejsu użytkownika oraz wskazanie propozycji testów. Praca nie koncentruje się na pełnej implementacji gry, lecz na przygotowaniu kompletnej dokumentacji projektowej, która może stanowić podstawę dalszej realizacji systemu.

1.2. Analiza problemu

Projektowanie gry *Godmist* wymaga rozwiązania problemu integracji wielu mechanizmów, które w praktyce muszą działać równocześnie i pozostawać ze sobą w zgodzie. W odróżnieniu od prostych aplikacji użytkowych, system gry RPG nie realizuje jednego procesu biznesowego, lecz obsługuje złożone środowisko interaktywne, w którym użytkownik może w dowolnym momencie zmieniać kontekst działania. Gracz porusza się pomiędzy eksploracją lokacji, zarządzaniem drużyną, walką, rozwojem postaci, craftowaniem przedmiotów, realizacją misji oraz zapisaniem aktualnego stanu gry. Z punktu widzenia inżynierii oprogramowania oznacza to konieczność zaprojektowania systemu odpornego na dużą liczbę stanów i przejść pomiędzy nimi.

Jednym z głównych wyzwań jest model postaci i drużyny. Każda postać posiada własne statystyki, klasę, zasób klasowy, umiejętności aktywne, pasywki, efekty statusu, ekwipunek oraz postęp w drzewkach rozwoju. Jednocześnie system musi rozróżniać stan trwały postaci od stanu tymczasowego

występującego w walce. Oznacza to potrzebę rozdzielenia definicji danych od runtime'owego stanu rozgrywki oraz właściwego modelowania relacji między nimi.

Drugim istotnym problemem jest walka turowa na siatce kwadratowej. Mechanika walki obejmuje kolejkę tur opartą na szybkości jednostek, punkty akcji, używanie umiejętności i przedmiotów, interakcję z obiektami mapy, system powalenia i śmierci, ucieczkę z walki oraz zależności pomiędzy wieloma drużynami. Oprócz samego modelu walki system musi poprawnie przenosić stan postaci pomiędzy eksploracją a starciem oraz zachowywać spójność danych po jego zakończeniu.

Kolejnym wyzwaniem jest model przedmiotów i progresji ekwipunku. W projekcie występują zarówno zwykłe przedmioty użytkowe, jak i złożone elementy wyposażenia generowane z części, posiadające jakość, rzadkość, poziom, możliwość ulepszania, infuzji, przekuwania oraz osadzania galdurytów. Oznacza to konieczność zaprojektowania elastycznej struktury danych, która z jednej strony pozwoli modelować różne typy przedmiotów, a z drugiej nie doprowadzi do nadmiernej komplikacji systemu.

Istotną trudność stanowi również powiązanie eksploracji z trwałością danych. Lochy proceduralne wymagają generatora, ale po wejściu do nich ich stan musi zostać zapamiętany przynajmniej do momentu opuszczenia instancji. Gra korzysta z zapisu w plikach JSON, a nie z klasycznej relacyjnej bazy danych, dlatego model danych musi umożliwiać czytelne przechowywanie stanu postaci, przedmiotów, misji, lochów, NPC i świata przy zachowaniu spójności identyfikatorów oraz relacji pomiędzy encjami.

Na poziomie architektury problemem jest także podział odpowiedzialności pomiędzy definicje danych, logikę domenową, serwisy oraz warstwę prezentacji. W projekcie zakłada się wykorzystanie silnika Unity, co sprzyja oddzieleniu definicji `ScriptableObject`, klas runtime opisujących stan gry, serwisów koordynujących logikę oraz klas `MonoBehaviour` odpowiedzialnych za widok, wejście użytkownika i integrację ze sceną. Poprawne rozdzielenie tych warstw jest warunkiem późniejszej rozbudowy projektu oraz utrzymania porządku w kodzie.

1.3. Przegląd istniejących rozwiązań

Projekt *Godmist* nie powstaje w oderwaniu od istniejących gier reprezentujących podobne mechaniki lub zbliżony sposób prowadzenia rozgrywki. Analiza wybranych rozwiązań pozwala wskazać elementy, które sprawdziły się w praktyce, a także lepiej określić własne założenia projektowe. Do porównania wybrano trzy tytuły: *Baldur's Gate 3*, *Darkest Dungeon* oraz *Clair Obscur: Expedition 33*. Każdy z nich reprezentuje inny aspekt projektowanego systemu.

1.3.1. Baldur's Gate 3

Baldur's Gate 3 jest rozbudowaną grą RPG opartą na zarządzaniu drużyną, eksploracji świata, interakcji z postaciami niezależnymi oraz realizacji zadań fabularnych i pobocznych. Jedną z najważniejszych cech tego rozwiązania jest silne powiązanie eksploracji z narracją i strukturą drużyny. Gracz nie steruje jedynie pojedynczym bohaterem, lecz zarządza grupą postaci, które posiadają własne role, umiejętności i znaczenie fabularne. Istotne jest także to, że część działań poza walką, takich jak rozmowy czy wybór aktywnej postaci, wpływa na dalszy przebieg rozgrywki.

Dla projektu *Godmist* inspirujące są przede wszystkim: model aktywnej drużyny, możliwość zmiany kontrolowanej postaci poza walką, obecność miasta lub huba pełniącego funkcję miejsca przygotowania do kolejnych wypraw oraz silne połączenie systemu misji z eksploracją i postaciami niezależnymi.

Podobnie jak w *Baldur's Gate 3*, w projekcie zakłada się, że gracz zarządza drużyną bohaterów, rozwija ich wyposażenie, realizuje zadania i podejmuje decyzje wpływające na stan świata.

Różnica polega jednak na tym, że *Godmist* nie jest klasycznym izometrycznym cRPG nastawionym w równym stopniu na narrację i swobodę dialogową. Większy nacisk położono tutaj na eksplorację lochów, powtarzalny cykl wypraw do niebezpiecznych miejsc i rozwój systemów związanych z walką oraz przedmiotami. Można więc uznać, że projekt przejmuje z *Baldur's Gate 3* logikę drużyny, miasta i zadań, ale rozwija ją w kierunku bardziej systemowego dungeon crawlera.

1.3.2. Darkest Dungeon

Darkest Dungeon stanowi jeden z najważniejszych punktów odniesienia dla projektu *Godmist*, ponieważ również opiera się na cyklu przygotowania drużyny, wyprawy do lochu, walki, zdobywania nagród i powrotu do miejsca pełniącego funkcję huba. Kluczową cechą tego typu gier jest napięcie wynikające z ryzyka wyprawy. Gracz musi podejmować decyzje dotyczące składu drużyny, wyposażenia, użycia zasobów oraz opłacalności dalszej eksploracji. Istotna staje się nie tylko sama walka, ale również zarządzanie konsekwencjami wcześniejszych decyzji.

W projekcie *Godmist* podobna jest sama struktura rozgrywki. Gracz przygotowuje drużynę w mieście, korzysta z usług NPC, a następnie udaje się do lochu, gdzie eksploruje pomieszczenia, uruchamia pułapki, walczy z przeciwnikami, zbiera łup i stara się bezpiecznie wrócić. W obu rozwiązaniach ważna jest także możliwość utraty części postępu lub postaci oraz konieczność inwestowania zdobytych zasobów w dalszy rozwój.

Jednocześnie *Godmist* różni się od *Darkest Dungeon* sposobem prowadzenia eksploracji i walki. Zamiast modelu bardziej abstrakcyjnego, projekt zakłada poruszanie się po mapie 2D złożonej z pokoi i korytarzy oraz walkę odbywającą się na tej samej siatce pól, bez przejścia do całkowicie osobnego ekranu walki. Dodatkowo rozwój postaci i ekwipunku w *Godmist* ma większy nacisk na klasy, drzewka umiejętności, części wyposażenia i system craftingu. Oznacza to, że inspirowana została przede wszystkim struktura wypraw i zarządzania ryzykiem, natomiast sama implementacja podsystemów jest bardziej rozbudowana.

1.3.3. Clair Obscur: Expedition 33

Trzecim istotnym punktem odniesienia jest *Clair Obscur: Expedition 33*. Tytuł ten stanowi dobrą inspirację przede wszystkim w obszarze rozwoju postaci, personalizacji buildów oraz kompozycji drużyny. W tego typu rozwiązaniach ważne jest, aby każda postać posiadała odrębny styl gry, własne ścieżki rozwoju oraz zestaw umiejętności pozwalających budować synergie z innymi członkami zespołu. Gracz powinien odczuwać, że rozwój nie polega jedynie na zwiększaniu liczb, ale również na kształtowaniu roli postaci w drużynie.

W *Godmist* podobny cel realizowany jest przez system klas, zasobów klasowych, aktywnych umiejętności, pasywek oraz drzewek skilli podzielonych na trzy niezależne ścieżki rozwoju. Każda klasa posiada własną tożsamość mechaniczną, własny zasób i inny zestaw możliwych buildów. Taki model wspiera różnorodność stylów gry oraz pozwala na świadome budowanie drużyny pod konkretną strategię.

Różnica polega na tym, że *Clair Obscur: Expedition 33* nie jest grą skoncentrowaną na wyprawach do proceduralnych lochów, zarządzaniu miastem czy generowanym ekwipunku. W projekcie

Godmist element progresji postaci został połączony z bardziej rozbudowanym systemem lootu, craftingu i eksploracji. Oznacza to, że inspiracja dotyczy głównie jakości rozwoju postaci, a nie ogólnej struktury całej gry.

1.3.4. Wnioski z analizy porównawczej

Przeprowadzona analiza pokazuje, że projekt *Godmist* łączy elementy pochodzące z kilku różnych nurtów gier RPG i dungeon crawlerów. Z *Baldur's Gate 3* przejmuje ideę zarządzania drużyną, eksploracji świata, interakcji z NPC i systemu misji. Z *Darkest Dungeon* czerpie strukturę wypraw do lochów, znaczenie przygotowania do wyprawy, napięcie związane z ryzykiem oraz rolę miasta jako centrum zaplecza. Z kolei *Clair Obscur: Expedition 33* stanowi inspirację w obszarze progresji postaci, różnicowania klas i tworzenia wyraźnych ścieżek rozwoju.

Na tle tych rozwiązań *Godmist* stanowi próbę połączenia eksploracji proceduralnych lochów, walki turowej, craftingu, systemu przedmiotów i rozwoju drużyny w jednym spójnym systemie. Własną cechą projektu jest połączenie podejścia dungeon crawlerowego z rozbudowaną strukturą danych i modularną architekturą, co czyni go dobrym obiektem analizy z perspektywy inżynierii oprogramowania.

1.4. Opis biznesowy systemu

Choć projekt *Godmist* dotyczy gry single player, możliwe jest opisanie go jako systemu informatycznego dostarczającego użytkownikowi określony zestaw funkcji. System ma umożliwiać prowadzenie rozgrywki polegającej na eksploracji miasta i lochów, zarządzaniu drużyną, walce, rozwoju postaci oraz trwałym zapisie postępu. Głównym użytkownikiem końcowym systemu jest gracz, który korzysta z udostępnionych funkcji w celu realizacji zadań, zdobywania zasobów i rozwijania drużyny.

System powinien być zaprojektowany w sposób modułowy, ponieważ obejmuje wiele współpracujących obszarów funkcjonalnych. Z perspektywy użytkownika istotne są nie tylko pojedyncze mechaniki, ale również płynne przechodzenie pomiędzy nimi. Gracz powinien mieć możliwość przygotowania drużyny w mieście, wyruszenia do lochu, prowadzenia eksploracji, rozpoczynania starć, zdobywania łupów, powrotu do miasta oraz dalszego inwestowania zdobytych zasobów. Tego typu cykl rozgrywki wymaga logicznie spójnego systemu zarządzania stanem gry.

1.4.1. Wymagania funkcjonalne

Wymagania funkcjonalne określają, jakie operacje i usługi system *Godmist* powinien realizować z perspektywy użytkownika oraz logiki rozgrywki. Poniższa lista obejmuje kluczowe funkcje projektowanego systemu.

- **WF-01.** System ma umożliwiać rozpoczęcie nowej gry oraz utworzenie nowego stanu kampanii.
- **WF-02.** System ma umożliwiać wczytanie wcześniej zapisanego stanu gry.
- **WF-03.** System ma umożliwiać zapis postępu gry w sposób ręczny oraz automatyczny w określonych momentach rozgrywki.
- **WF-04.** System ma umożliwiać poruszanie drużyną po mapie miasta oraz po mapach lochów.
- **WF-05.** System ma umożliwiać przełączanie aktywnie kontrolowanej postaci poza walką pomiędzy członkami drużyny kontrolowanymi przez gracza.

- **WF-06.** System ma umożliwiać interakcję z obiektami środowiskowymi, takimi jak drzwi, skrzynie, dźwignie, pułapki i inne obiekty interaktywne.
- **WF-07.** System ma umożliwiać rozpoczynanie dialogów z postaciami niezależnymi.
- **WF-08.** System ma umożliwiać inicjowanie walki na skutek wejścia w obszar starcia, interakcji skryptowej lub określonego wyboru dialogowego.
- **WF-09.** System ma umożliwiać prowadzenie walki w trybie turowym na planszy podzielonej na pola siatki kwadratowej.
- **WF-10.** System ma wyznaczać kolejność tur na podstawie parametrów postaci i stanu bieżącej walki.
- **WF-11.** System ma umożliwiać wykonywanie ruchu postaci po planszy w trakcie walki z uwzględnieniem kosztu punktów akcji.
- **WF-12.** System ma uwzględniać wpływ terenu i przeszkód na możliwość ruchu, koszt ruchu oraz dostępność wybranych pól.
- **WF-13.** System ma umożliwiać używanie umiejętności aktywnych podczas walki.
- **WF-14.** System ma uwzględniać koszt punktów akcji oraz koszt zasobu klasowego podczas używania umiejętności.
- **WF-15.** System ma obsługiwać różne typy celowania umiejętności, w tym cele pojedyncze, obszarowe, liniowe, łańcuchowe oraz niestandardowe.
- **WF-16.** System ma obsługiwać różne efekty umiejętności, w tym zadawanie obrażeń, leczenie, nakładanie statusów, modyfikację statystyk, przesunięcia, teleportację, tarczę, przywracanie zasobów i tworzenie obiektów.
- **WF-17.** System ma umożliwiać używanie przedmiotów bojowych i użytkowych podczas walki.
- **WF-18.** System ma umożliwiać próbę ucieczki z walki zgodnie z przyjętymi regułami mechaniki.
- **WF-19.** System ma umożliwiać podnoszenie powalonych postaci przez uprawnione jednostki podczas walki.
- **WF-20.** System ma obsługiwać zakończenie walki wraz z odpowiednim rozstrzygnięciem wyniku starcia.
- **WF-21.** System ma obsługiwać statystyki postaci, w tym zdrowie, atak, obronę fizyczną i magiczną, unik, szansę trafienia krytycznego, obrażenia krytyczne, szybkość oraz regenerację zasobu.
- **WF-22.** System ma obsługiwać klasowe zasoby postaci, takie jak rage, momentum i mana, wraz z ich regeneracją i zużyciem.

- **WF-23.** System ma obsługiwać statusy i efekty czasowe, w tym efekty pozytywne, negatywne i neutralne.
- **WF-24.** System ma obsługiwać odporności postaci na wybrane rodzaje efektów i ograniczeń.
- **WF-25.** System ma umożliwiać zdobywanie doświadczenia przez postacie oraz rozwój poziomu postaci.
- **WF-26.** System ma umożliwiać przydzielanie punktów rozwoju do drzewek umiejętności przypisanych do klasy postaci.
- **WF-27.** System ma obsługiwać odblokowywanie aktywnych i pasywnych umiejętności na podstawie postępu w drzewkach rozwoju.
- **WF-28.** System ma umożliwiać zarządzanie ekwipunkiem postaci, w tym przenoszenie, przechowywanie, zużywanie i sprzedawanie przedmiotów.
- **WF-29.** System ma obsługiwać wspólny schowek drużyny dostępny w mieście.
- **WF-30.** System ma umożliwiać wyposażanie postaci w broń, pancerz i biżuterię zgodnie z ograniczeniami klasowymi i poziomowymi.
- **WF-31.** System ma obsługiwać przedmioty instancyjne z własnymi cechami, takimi jak poziom, rzadkość, jakość, ulepszenie oraz efekty dodatkowe.
- **WF-32.** System ma obsługiwać system galdurytów, ich identyfikację, osadzanie w wyposażeniu oraz wynikające z tego efekty pasywne.
- **WF-33.** System ma umożliwiać tworzenie przedmiotów u odpowiednich postaci niezależnych, w tym broni, pancerzy, biżuterii, mikstur, bomb, materiałów pośrednich i galdurytów.
- **WF-34.** System ma umożliwiać ulepszanie, przekuwanie oraz infuzję wybranych elementów wyposażenia.
- **WF-35.** System ma obsługiwać handel z postaciami niezależnymi, w tym kupowanie, sprzedawanie i odkup przedmiotów.
- **WF-36.** System ma obsługiwać lojalność wybranych postaci niezależnych i jej wpływ na dostępność usług oraz ceny.
- **WF-37.** System ma obsługiwać parametr honoru postaci oraz jego wpływ na wybrane interakcje społeczne i ekonomiczne.
- **WF-38.** System ma umożliwiać przyjmowanie, śledzenie i realizowanie zadań głównych, pobocznych, towarzyszy, frakcyjnych i radiant.
- **WF-39.** System ma obsługiwać wieloetapowe zadania wraz z celami, warunkami dostępności, nagrodami i flagami świata.
- **WF-40.** System ma umożliwiać generowanie proceduralnych lochów na podstawie ustalonych parametrów, motywów oraz reguł zawartości.

- **WF-41.** System ma obsługiwać lochy predefiniowane oraz lochy częściowo proceduralne zawierające gwarantowane pomieszczenia i zdarzenia.
- **WF-42.** System ma zapamiętywać stan odwiedzonych lochów, w tym układ, odwiedzone pomieszczenia, pokonane spotkania oraz zdobyty łup.
- **WF-43.** System ma obsługiwać postacie niezależne świadczące różne typy usług, takie jak handel, kowalstwo, alchemia, runotwórstwo i zadania.
- **WF-44.** System ma umożliwiać wyświetlanie odpowiednich widoków interfejsu użytkownika dla eksploracji, walki, ekwipunku, rozwoju, craftingu, handlu, questów i zapisu gry.
- **WF-45.** System ma umożliwiać zarządzanie wieloma zapisami gry w formie listy kampanii i stanów zapisu.

1.4.2. Wymagania niefunkcjonalne

Wymagania niefunkcjonalne określają ograniczenia jakościowe, technologiczne i organizacyjne projektowanego systemu. Dotyczą one sposobu działania systemu, jego wydajności, architektury oraz wygody użytkowania.

- **WN-01.** System ma działać jako aplikacja typu single player, niewymagająca logowania użytkownika ani stałego połączenia z Internetem.
- **WN-02.** System ma być rozwijany z wykorzystaniem silnika Unity.
- **WN-03.** System ma być implementowany w języku C#.
- **WN-04.** System ma przechowywać trwały stan gry w plikach w formacie JSON.
- **WN-05.** Architektura systemu ma rozdzielać definicje danych, stan runtime, logikę systemową, warstwę prezentacji oraz mechanizmy trwałości danych.
- **WN-06.** System ma umożliwiać dalszą rozbudowę o nowe klasy postaci, umiejętności, przedmioty, questy, NPC oraz lochy bez konieczności przebudowy całej architektury.
- **WN-07.** System ma wykorzystywać identyfikatory logiczne w sposób zapewniający spójne odwoływanie się do definicji danych i instancji zapisu.
- **WN-08.** System ma zachowywać spójność danych po zapisaniu i ponownym wczytaniu gry.
- **WN-09.** System ma uniemożliwiać wykonanie operacji prowadzących do oczywistej niespójności stanu rozgrywki, w szczególności w obszarach walki, ekwipunku, questów i zapisu.
- **WN-10.** Interfejs użytkownika ma być czytelny i dostosowany do obsługi z użyciem myszy oraz klawiatury.
- **WN-11.** Interfejs użytkownika ma zachowywać spójność wizualną i funkcjonalną pomiędzy głównymi ekranami systemu.

- **WN-12.** System ma dostarczać użytkownikowi jednoznacznej informacji o wynikach wykonywanych operacji, takich jak zapis gry, zakup przedmiotu, użycie umiejętności lub brak spełnienia warunków działania.
- **WN-13.** System ma zapewniać akceptowalny czas reakcji podczas przełączania widoków, otwierania okien interfejsu oraz wykonywania podstawowych operacji użytkownika.
- **WN-14.** System ma zapewniać akceptowalny czas generowania i wczytywania lochów w typowych scenariuszach użytkowania.
- **WN-15.** System ma zachowywać stabilność działania podczas dłuższej sesji gry obejmującej wielokrotne walki, interakcje, handel i zapisywanie stanu.
- **WN-16.** System ma umożliwiać utrzymanie i testowanie poszczególnych modułów w sposób względnie niezależny.
- **WN-17.** System ma być przygotowany do stosowania podejścia zdarzeniowego w komunikacji pomiędzy wybranymi modułami.
- **WN-18.** System ma umożliwiać tworzenie narzędzi debugowych i edytorowych wspierających rozwój projektu w środowisku Unity.
- **WN-19.** System ma być odporny na częściowe uszkodzenie pojedynczego zapisu w tym sensie, że uszkodzenie jednego pliku zapisu nie powinno uniemożliwiać korzystania z pozostałych zapisów.
- **WN-20.** System ma umożliwiać rozwój dokumentacji projektowej i modelu klas w sposób zgodny z rzeczywistą architekturą implementacji.
- **WN-21.** System ma wspierać modularne dodawanie nowych danych konfiguracyjnych z wykorzystaniem obiektów `ScriptableObject`.
- **WN-22.** System ma być możliwy do uruchomienia na typowym komputerze osobistym klasy konsumenckiej, spełniającym wymagania środowiska uruchomieniowego Unity.
- **WN-23.** System ma zapewniać możliwość dalszego rozwinięcia warstwy fabularnej, zawartości lochów oraz złożoności mechanik bez konieczności zmiany podstawowych założeń technologicznych.
- **WN-24.** Nazewnictwo elementów systemu, struktur danych i modułów powinno być spójne w kodzie, dokumentacji i modelach projektowych.
- **WN-25.** System powinien być projektowany w sposób ograniczający nadmierne uzależnienie logiki domenowej od klas widoku i elementów sceny Unity.

1.5. Podsumowanie rozdziału

W rozdziale przedstawiono cel i zakres pracy, omówiono problem projektowy oraz dokonano przeglądu wybranych istniejących rozwiązań. Na tej podstawie określono również podstawowe wymagania funkcjonalne i niefunkcjonalne projektowanego systemu. Ustalenia te stanowią punkt wyjścia do dalszego modelowania, projektowania architektury oraz opisu struktury danych gry *Godmist*.

2. Projekt systemu i struktura rozwiązania

2.1. Modelowanie systemu

Modelowanie systemu stanowi jeden z kluczowych etapów projektowania gry *Godmist* z perspektywy inżynierii oprogramowania. Ze względu na złożoność projektowanego rozwiązania konieczne jest opisanie zarówno zachowania systemu z punktu widzenia użytkownika, jak i jego struktury wewnętrznej. W tym celu zastosowano wybrane diagramy języka UML, które pozwalają uporządkować zależności pomiędzy poszczególnymi elementami projektu oraz ułatwiają późniejszą implementację.

W przypadku systemu *Godmist* modelowanie obejmuje kilka wzajemnie uzupełniających się perspektyw. Pierwszą z nich jest perspektywa funkcjonalna, pokazująca, jakie działania może wykonywać użytkownik oraz jakie funkcje system powinien udostępniać. Drugą perspektywą jest opis przebiegu wybranych procesów, takich jak walka, eksploracja lochu, crafting lub zapis gry. Trzecią perspektywę stanowi model zachowania poszczególnych obiektów, na przykład postaci lub misji, których stan zmienia się w czasie rozgrywki.

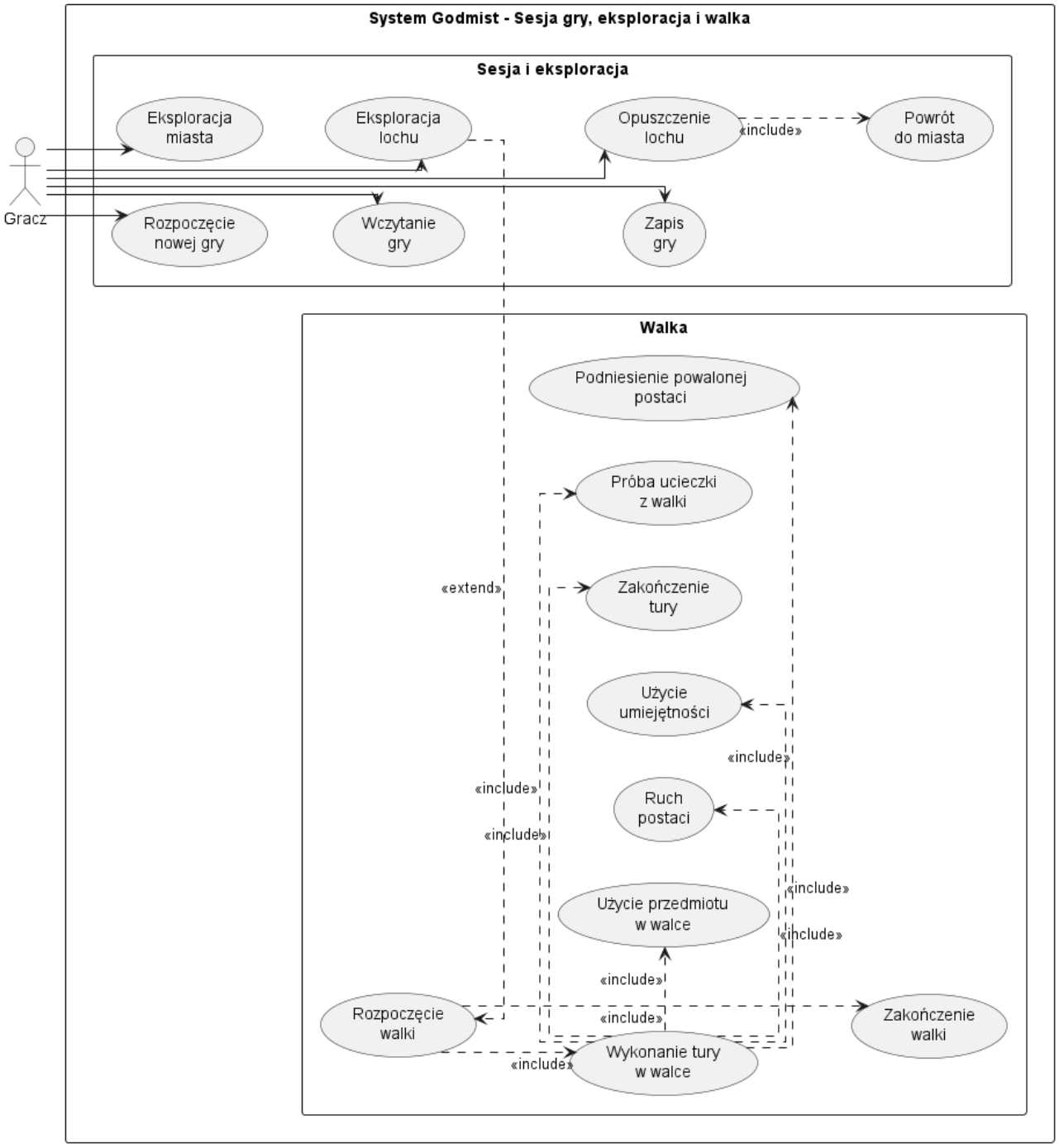
Zastosowanie diagramów UML w projekcie pozwala przedstawić logikę działania systemu w sposób bardziej formalny i czytelny. Dzięki temu możliwe jest nie tylko lepsze zrozumienie zależności pomiędzy modułami gry, ale również weryfikacja spójności założeń jeszcze przed pełną implementacją. W przypadku gry obejmującej wiele mechanik, takich jak rozwój postaci, walka turowa, eksploracja proceduralnych lochów, zarządzanie ekwipunkiem, interakcje z NPC oraz system misji, etap modelowania jest szczególnie istotny.

W kolejnych podsekcjach przedstawione zostaną wybrane modele systemu: diagram przypadków użycia, scenariusze przypadków użycia, diagramy aktywności, diagramy sekwencji oraz diagramy stanów.

2.1.1. Diagramy przypadków użycia

Ze względu na szeroki zakres funkcjonalny systemu *Godmist* oraz dużą liczbę relacji pomiędzy przypadkami użycia, zrezygnowano z przedstawiania wszystkich funkcji w postaci jednego zbiorczego diagramu. Zamiast tego zastosowano podział na trzy mniejsze diagramy przypadków użycia, odpowiadające głównym obszarom logiki systemu. Takie podejście pozwala zachować czytelność modeli, a jednocześnie nie powoduje utraty istotnych zależności pomiędzy funkcjami.

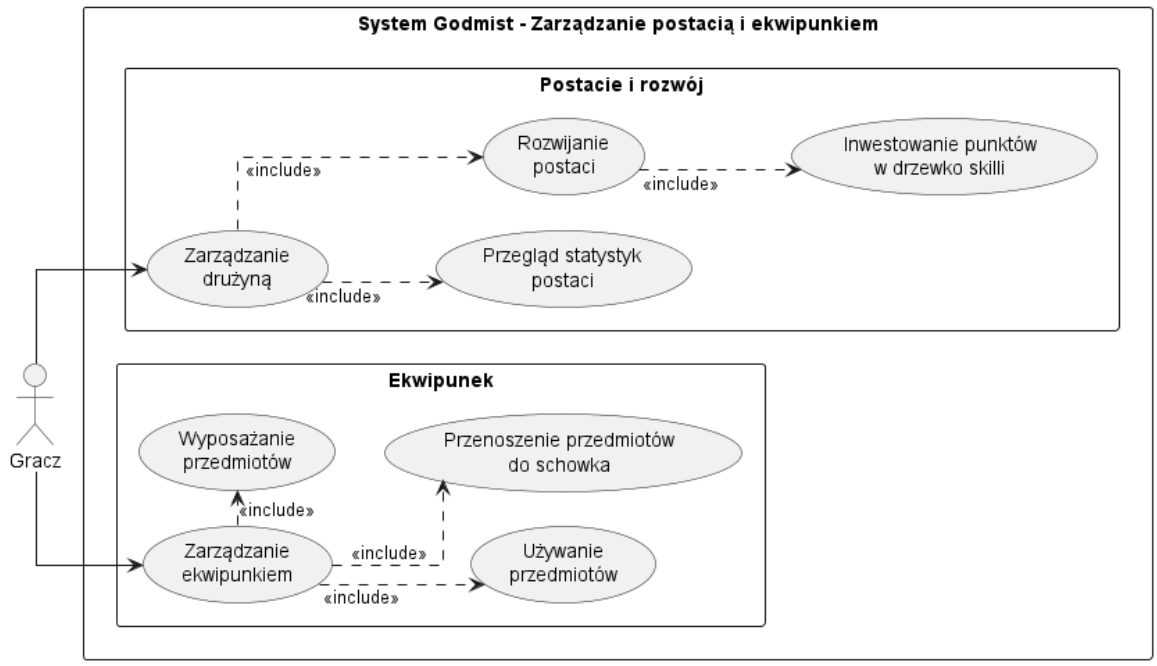
Pierwszy diagram obejmuje obsługę sesji gry, eksplorację oraz walkę. Drugi koncentruje się na zarządzaniu postacią i ekwipunkiem, natomiast trzeci przedstawia funkcje związane z interakcjami z NPC, handlem, craftingiem oraz obsługą questów. Łącznie trzy diagramy tworzą spójny obraz funkcjonalny projektowanego systemu z perspektywy głównego aktora, którym jest gracz.



Rys. 2.1. Diagram przypadków użycia dla sesji gry, eksploracji i walki.

Źródło: Opracowanie własne

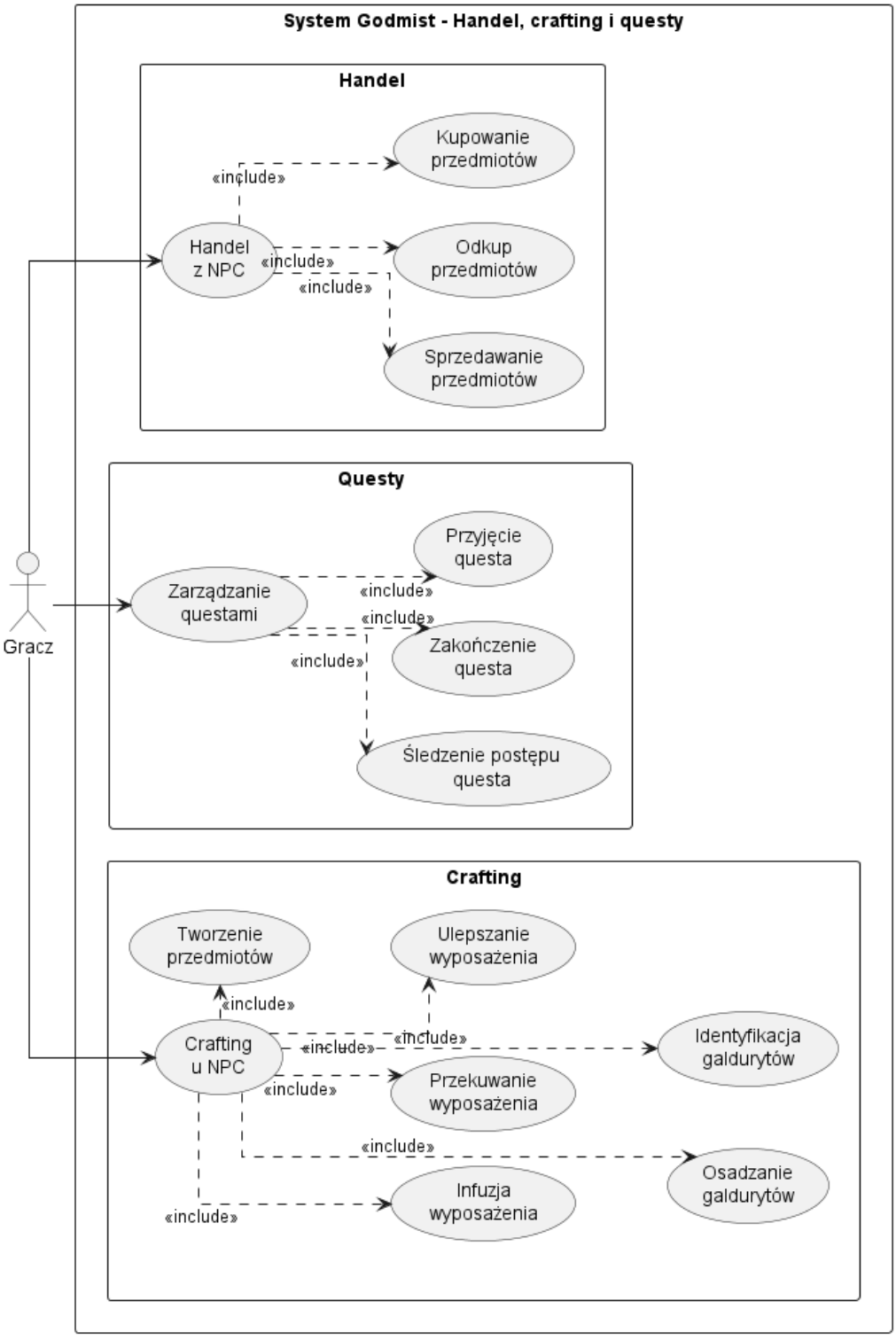
Pierwszy diagram przedstawia funkcje związane z rozpoczęciem i obsługą sesji gry, eksploracją miasta i lochu oraz przejściem do systemu walki. Szczególnie istotna jest tu relacja *extend* pomiędzy eksploracją lochu a rozpoczęciem walki, ponieważ pokazuje ona, że walka stanowi zdarzenie uruchamiane w określonym kontekście eksploracji. Diagram obejmuje również podstawowe działania wykonywane w trakcie starcia oraz zakończenie walki i powrót z lochu do miasta.



Rys. 2.2. Diagram przypadków użycia dla zarządzania postacią i ekwipunkiem.

Źródło: Opracowanie własne

Drugi diagram koncentruje się na funkcjach związanych z zarządzaniem drużyną, rozwojem postaci oraz obsługą ekwipunku. Pokazano na nim zależności pomiędzy przeglądem statystyk, rozwijaniem postaci i inwestowaniem punktów w drzewka skilli, a także podstawowe operacje wykonywane na przedmiotach. Diagram ten przedstawia obszar systemu odpowiedzialny za rozwój drużyny oraz organizację zasobów gracza poza bezpośrednim przebiegiem walki.



Rys. 2.3. Diagram przypadków użycia dla handlu, craftingu i questów.

Źródło: Opracowanie własne

Trzeci diagram obejmuje funkcje realizowane głównie w interakcji z postaciami niezależnymi, a więc handel, crafting oraz zarządzanie questami. Zostały na nim przedstawione zarówno podstawowe operacje handlowe, jak i bardziej rozbudowane działania związane z ulepszaniem wyposażenia, infuzją oraz obsługą galdurytów. Ujęcie questów w tym samym diagramie wynika z ich silnego powiązania z interakcjami z NPC oraz miejskim zapleczem rozgrywki.

2.1.2. Scenariusze przypadków użycia

W niniejszym podrozdziale przedstawiono dwa przykładowe scenariusze przypadków użycia dla systemu *Godmist*. Wybrane scenariusze obejmują jeden przypadek związany bezpośrednio z rozgrywką oraz jeden przypadek o charakterze systemowym. Pozwala to pokazać zarówno logikę mechanik gry, jak i obsługę trwałości danych.

Scenariusz przypadku użycia: Rozpoczęcie walki

Nazwa przypadku użycia: Rozpoczęcie walki

Aktor główny: Gracz

Cel: Rozpoczęcie starcia pomiędzy drużyną gracza a przeciwnikami w wyniku eksploracji, interakcji skryptowej lub dialogu.

Warunki wstępne:

- gra znajduje się w stanie aktywnej eksploracji miasta lub lochu,
- na mapie istnieje grupa jednostek lub zdarzenie mogące wywołać walkę,
- stan gry pozwala na przejście do trybu walki.

Warunki końcowe:

- zostaje utworzona aktywna sesja walki,
- uczestnicy walki zostają przypisani do odpowiednich drużyn i relacji,
- zostaje wyznaczona kolejność tur,
- interfejs użytkownika przechodzi do trybu walki.

Przebieg główny:

1. Gracz porusza drużyną po mapie lub podejmuje interakcję prowadzącą do starcia.
2. System wykrywa spełnienie warunku rozpoczęcia walki.
3. System identyfikuje jednostki biorące udział w starciu.
4. System wyznacza obszar walki na podstawie miejsca kontaktu i aktualnej mapy.
5. System tworzy instancję sesji walki.
6. System przekształca uczestników zdarzenia w jednostki bojowe wraz z ich bieżącym stanem.
7. System przypisuje uczestników do odpowiednich drużyn oraz określa relacje pomiędzy nimi.
8. System inicjalizuje parametry walki, w tym pozycje jednostek, aktualne punkty akcji i zasoby.
9. System wyznacza kolejność tur zgodnie z obowiązującymi regułami.
10. System przełącza interfejs użytkownika do trybu walki.

11. System przekazuje sterowanie pierwszej jednostce w kolejności tury.

Przebiegi alternatywne:

- Jeżeli walka została wywołana przez dialog, system rozpoczyna starcie po zakończeniu wybranej kwestii dialogowej.
- Jeżeli na planszy występują neutralne lub sojusznicze jednostki niezależne, system może dodać je do walki zgodnie z relacjami do drużyn.
- Jeżeli starcie przestaje być zasadne jeszcze przed właściwą inicjalizacją, system nie tworzy sesji walki i pozostaje w trybie eksploracji.

Uwagi:

- rozpoczęcie walki nie przenosi gracza do oddzielnej mapy, lecz wykorzystuje tę samą przestrzeń podzieloną na pola,
- pole walki zależy od aktualnego otoczenia, na przykład korytarza, pomieszczenia lub otwartego terenu.

Scenariusz przypadku użycia: Zapis gry

Nazwa przypadku użycia: Zapis gry
Aktor główny: Gracz
Cel: Utrwalenie aktualnego stanu gry w pliku zapisu, tak aby można było go później odtworzyć.
Warunki wstępne:

- gra znajduje się w stanie, w którym zapis jest dozwolony,
- istnieje aktywna kampania lub aktywny stan rozgrywki,
- system posiada dostęp do lokalizacji przeznaczonej na pliki zapisu.

Warunki końcowe:

- aktualny stan gry zostaje zapisany w nowym lub zaktualizowanym wpisie listy zapisów,
- metadane zapisu zostają zaktualizowane,
- użytkownik otrzymuje informację o powodzeniu lub niepowodzeniu operacji.

Przebieg główny:

1. Gracz wybiera opcję zapisu gry z poziomu interfejsu systemowego.
2. System weryfikuje, czy aktualny stan gry pozwala na wykonanie zapisu.
3. System pobiera bieżący stan postaci, świata, questów, lochów, NPC, schowka i innych istotnych elementów rozgrywki.
4. System przekształca dane runtime do struktur przeznaczonych do serializacji.

5. System buduje kompletny obiekt danych zapisu gry.
6. System serializuje dane do formatu JSON.
7. System zapisuje dane do pliku.
8. System aktualizuje metadane zapisu, takie jak data, nazwa lub identyfikator zapisu.
9. System odświeża listę dostępnych zapisów.
10. System informuje gracza o poprawnym zakończeniu operacji.

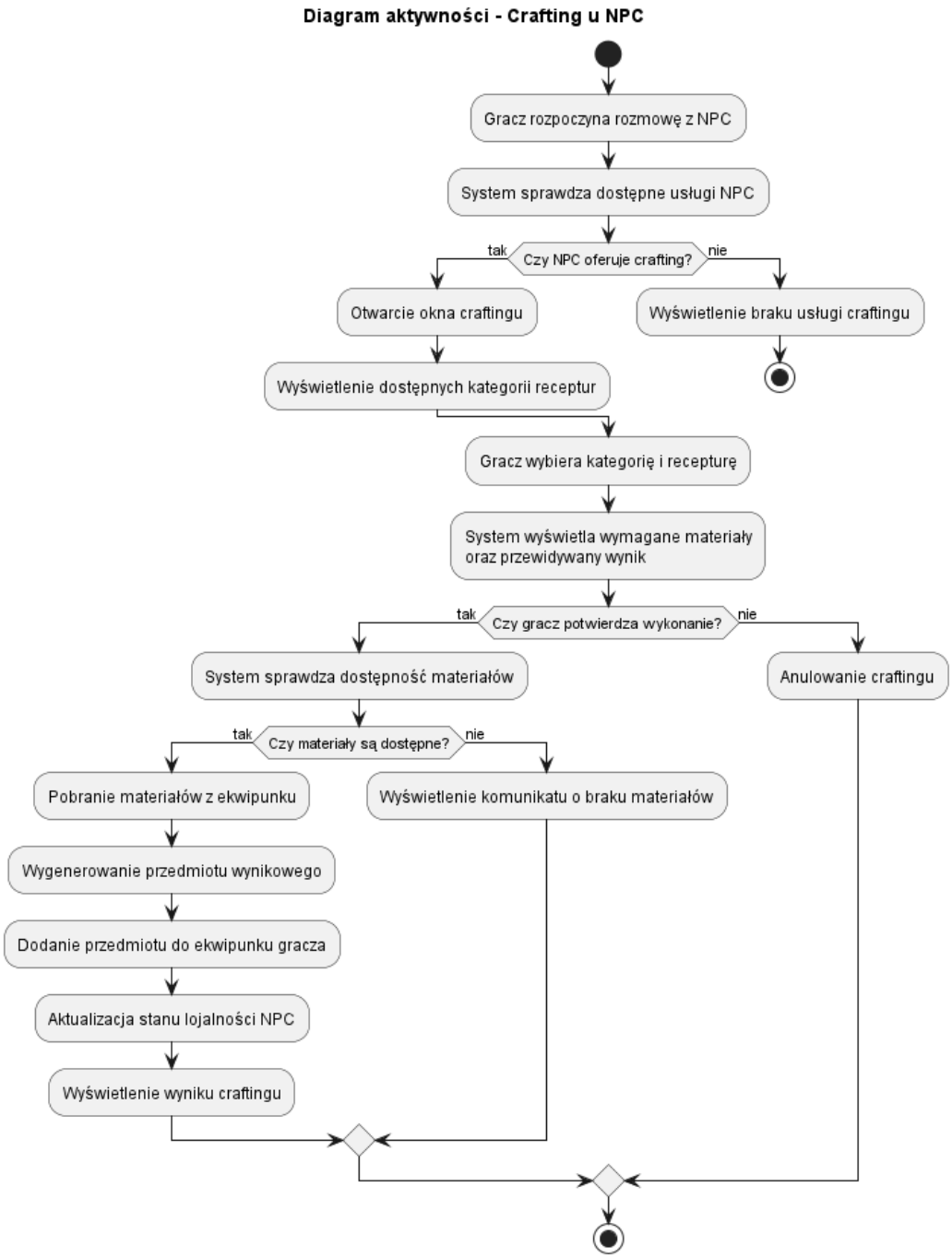
Przebiegi alternatywne:

- Jeżeli zapis gry jest niedozwolony w aktualnym stanie, system przerywa operację i wyświetla odpowiedni komunikat.
- Jeżeli podczas serializacji lub zapisu pliku wystąpi błąd, system zgłasza niepowodzenie operacji i nie nadpisuje poprawnych wcześniejszych zapisów.
- Jeżeli zapis wykonywany jest automatycznie, system pomija część interakcji użytkownika i zapisuje stan zgodnie z regułami autosave.

Uwagi:

- zapis gry obejmuje zarówno stan postaci i ich wyposażenia, jak i postęp questów, stan świata, lochów oraz informacje o usługach i ofertach NPC,
- model zapisu oparty jest na plikach JSON i nie wykorzystuje relacyjnej bazy danych.

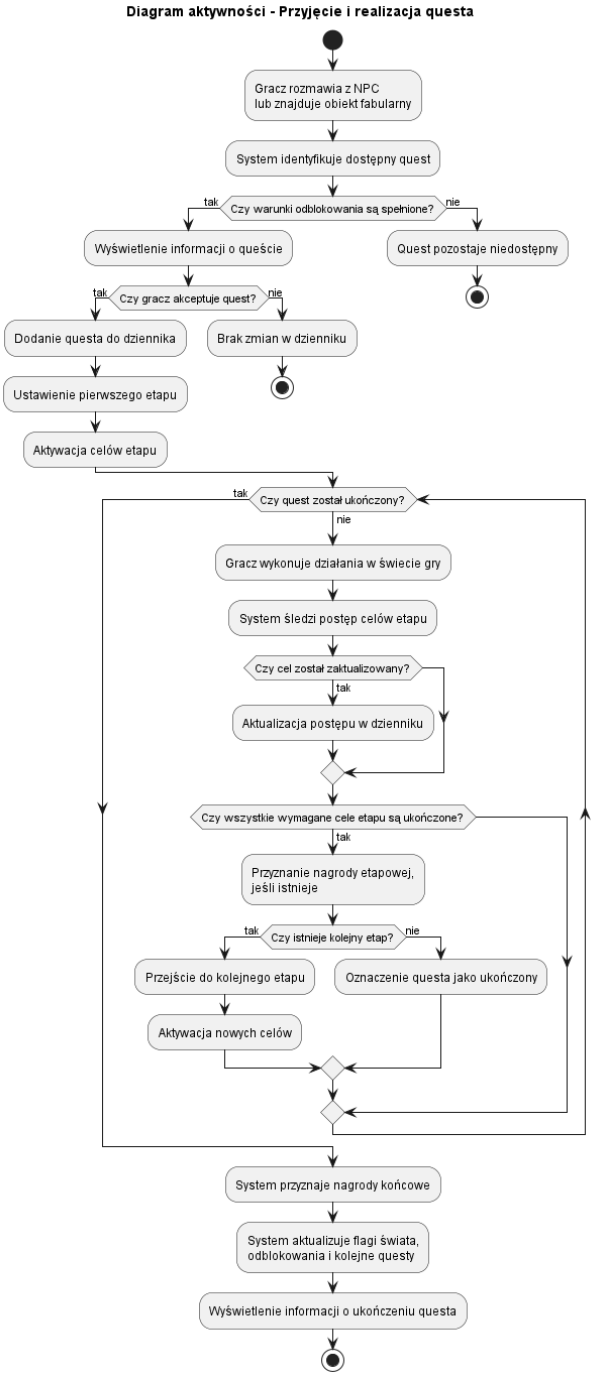
2.1.3. Diagramy aktywności



Rys. 2.4. Diagram aktywności przedstawiający proces craftingu u NPC.

Źródło: Opracowanie własne

Diagram przedstawia przebieg procesu tworzenia przedmiotów z perspektywy gracza i systemu. Uwzględniono w nim zarówno weryfikację dostępności usługi craftingu u danego NPC, jak i sprawdzenie wymaganych materiałów przed wykonaniem operacji. Diagram pokazuje również możliwe ścieżki alternatywne, takie jak brak odpowiedniej usługi lub anulowanie procesu przez gracza.

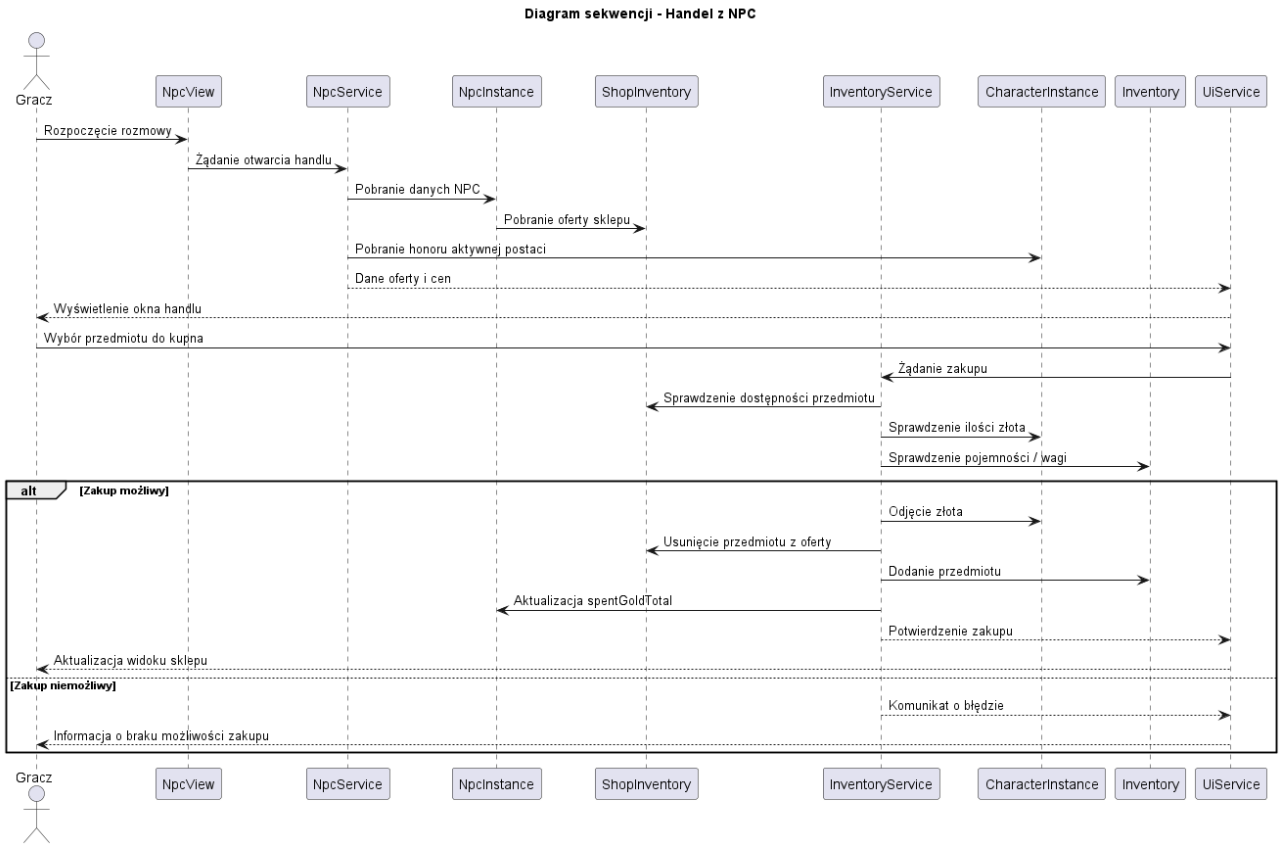


Rys. 2.5. Diagram aktywności przedstawiający przyjęcie i realizację questa.

Źródło: Opracowanie własne

Diagram opisuje pełny cykl życia zadania od momentu jego wykrycia do zakończenia i przyznania nagród. Pokazano na nim mechanizmy sprawdzania warunków odblokowania, akceptacji zadania przez gracza oraz aktualizacji postępu w kolejnych etapach. Ujęto także możliwość przechodzenia do następnych etapów oraz aktualizacji stanu świata po ukończeniu questa.

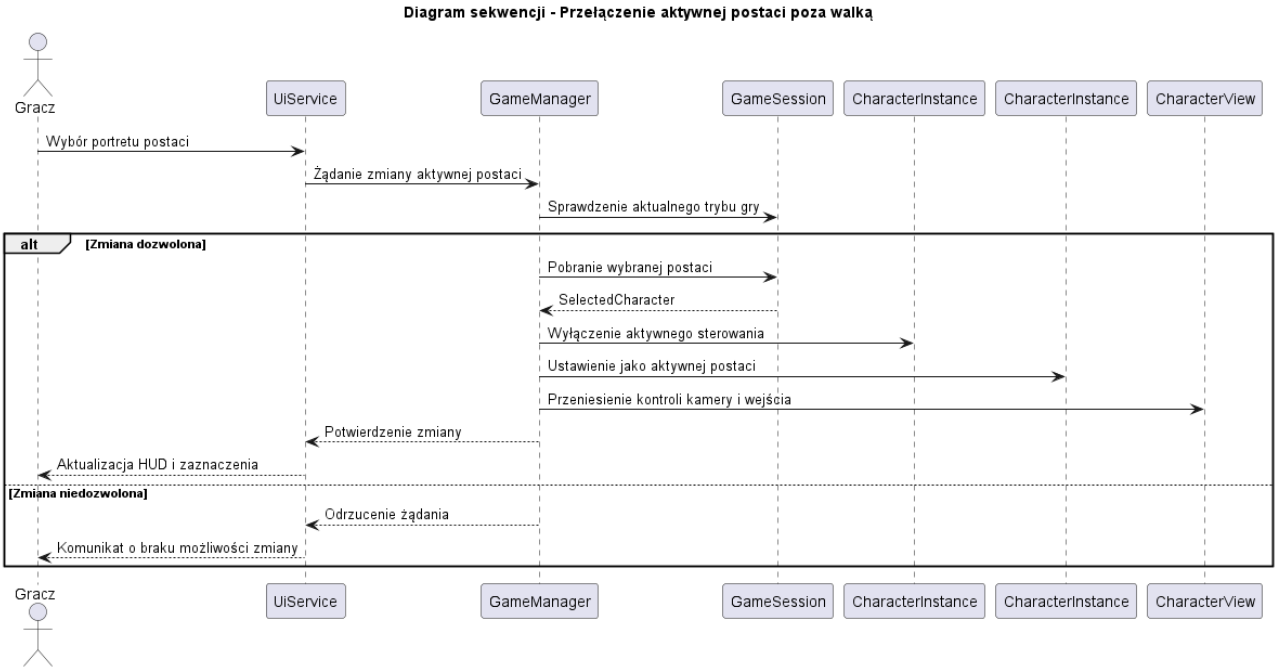
2.1.4. Diagramy sekwencji



Rys. 2.6. Diagram sekwencji przedstawiający handel z NPC.

Źródło: Opracowanie własne

Diagram sekwencji pokazuje współpracę pomiędzy interfejsem użytkownika, systemem NPC, ekwipunkiem oraz logiką zakupu podczas handlu. Proces rozpoczyna się od wejścia w interakcję z postacią niezależną i otwarcia okna handlu, a następnie przechodzi do weryfikacji oferty, ilości złota oraz ograniczeń ekwipunku. Diagram uwzględnia zarówno poprawny zakup, jak i sytuację, w której transakcja nie może zostać zrealizowana.

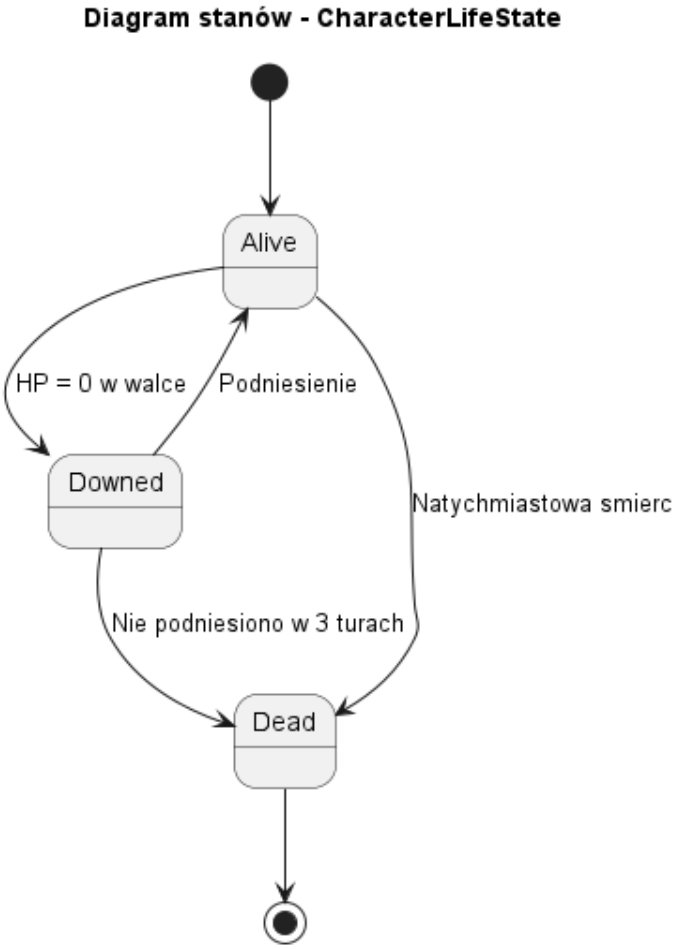


Rys. 2.7. Diagram sekwencji przedstawiający przełączenie aktywnej postaci poza walką.

Źródło: Opracowanie własne

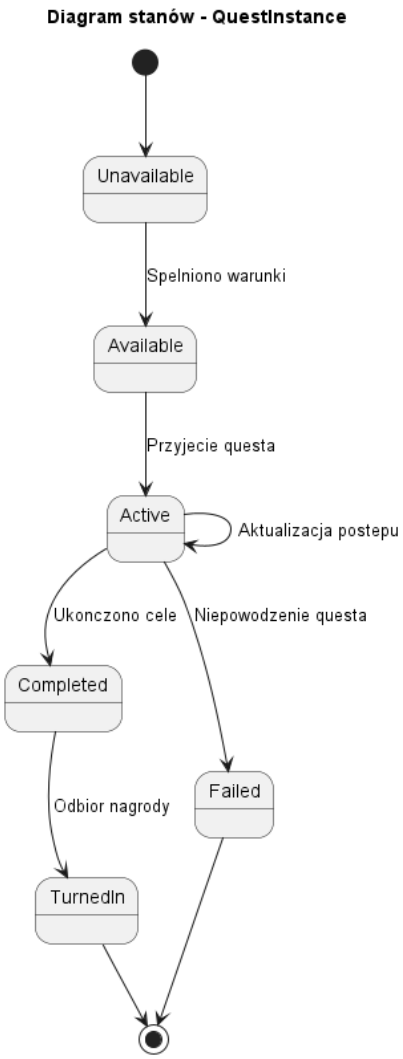
Diagram obrazuje proces zmiany aktualnie sterowanej postaci w trybie eksploracji. Pokazano w nim sprawdzenie, czy zmiana jest dozwolona w bieżącym stanie gry, a następnie przekazanie sterowania nowo wybranej postaci oraz aktualizację warstwy prezentacji. Uwzględniono również ścieżkę alternatywną, w której zmiana zostaje odrzucona przez system.

2.1.5. Diagramy stanów



Rys. 2.8. Diagram stanów obiektu CharacterLifeState.
Źródło: Opracowanie własne

Diagram stanów przedstawia możliwe stany życia postaci oraz warunki przejścia pomiędzy nimi. Model obejmuje stan aktywny, powalenie oraz śmierć definitywną, zgodnie z założeniami systemu walki w grze. Szczególnie istotne jest tu przejście ze stanu *Downed* do *Dead*, które następuje, gdy postać nie zostanie podniesiona w ciągu trzech tur.



Rys. 2.9. Diagram stanów obiektu QuestInstance.

Źródło: Opracowanie własne

Diagram opisuje zmiany stanu instancji zadania w trakcie jego udostępniania, realizacji i zamykania. Uwzględniono w nim etap odblokowania zadania, jego przyjęcie, aktualizację postępu, ukończenie oraz odbiór nagrody. Model przewiduje również możliwość niepowodzenia zadania, co pozwala odwzorować bardziej złożone przebiegi misji w systemie questów.

2.2. Technologie i wymagania sprzętowe

Wybór technologii zastosowanych w projekcie *Godmist* wynika z charakteru systemu oraz założonego zakresu funkcjonalnego. Projektowana aplikacja jest grą komputerową typu single player, łączącą eksplorację lochów, walkę turową, rozwój postaci, system przedmiotów, crafting, interakcje z NPC oraz trwały zapis postępu użytkownika. Oznacza to konieczność zastosowania narzędzi, które umożliwiają zarówno budowę warstwy wizualnej i interaktywnej, jak i czytelne modelowanie logiki systemu oraz trwałości danych.

W niniejszej sekcji przedstawiono główne technologie planowane do wykorzystania w projekcie oraz orientacyjne wymagania sprzętowe dla uruchomienia systemu. Ze względu na fakt, że projekt znajduje się na etapie dokumentacji i modelowania, część parametrów technicznych ma charakter założeń wstępnych i może zostać doprecyzowana na dalszych etapach implementacji.

2.2.1. Technologie

Podstawową technologią zastosowaną w projekcie jest silnik *Unity*. Został on wybrany ze względu na szerokie możliwości tworzenia gier 2D, dobrą integrację warstwy wizualnej z logiką systemu oraz wygodny model pracy oparty na scenach, komponentach i obiektach. Unity umożliwia również stosowanie wzorców architektonicznych wspierających rozdzielenie definicji danych, logiki runtime oraz warstwy prezentacji, co jest istotne z punktu widzenia utrzymania porządku w projekcie.

Językiem implementacji systemu jest *C#*. Wybór ten wynika bezpośrednio z integracji z silnikiem Unity, ale również z cech samego języka, takich jak obiektowy model programowania, dobra organizacja kodu, możliwość modelowania złożonych struktur danych oraz wygodne tworzenie klas domenowych, serwisów i warstw pośrednich. W projekcie zakłada się szerokie wykorzystanie klas reprezentujących zarówno definicje danych, jak i zmienny stan rozgrywki.

Ważnym elementem architektury systemu jest wykorzystanie obiektów `ScriptableObject`. Mechanizm ten pozwala przechowywać definicje statycznych danych gry, takich jak klasy postaci, definicje przedmiotów, umiejętności, questy czy definicje lochów. Dzięki temu możliwe jest oddzielenie danych konfiguracyjnych od runtime'owego stanu gry oraz łatwiejsze rozwijanie projektu bez konieczności wprowadzania wszystkich zmian bezpośrednio w kodzie źródłowym.

Do przechowywania stanu gry planowane jest zastosowanie plików *JSON*. Rozwiązanie to jest odpowiednie dla gry single player, w której nie ma potrzeby wykorzystywania zewnętrznej, relacyjnej bazy danych. Zapis w formacie JSON umożliwia przechowywanie stanu postaci, misji, NPC, lochów, ekwipunku i innych elementów w sposób strukturalny i łatwy do odtworzenia. Model ten dobrze współgra z projektowaną architekturą opartą na identyfikatorach definicji i klasach runtime.

W procesie projektowania wykorzystywane są również diagramy UML oraz narzędzia do modelowania wizualnego. Ich zadaniem jest uporządkowanie struktury systemu i przedstawienie zależności pomiędzy jego elementami jeszcze przed pełną implementacją. Dzięki temu możliwe jest wczesne wykrywanie niespójności oraz bardziej świadome planowanie podziału systemu na moduły.

Można zatem przyjąć, że główny stos technologiczny projektu obejmuje:

- silnik *Unity*,
- język programowania *C#*,
- definicje danych oparte na `ScriptableObject`,
- zapis stanu gry w plikach *JSON*,
- modelowanie projektowe z użyciem diagramów UML.

2.2.2. Wymagania sprzętowe

Projektowany system przeznaczony jest do uruchamiania na komputerach osobistych. Ze względu na charakter gry 2D oraz brak rozbudowanych wymagań związanych z renderingiem trójwymiarowym, nie przewiduje się bardzo wysokich wymagań sprzętowych. Jednocześnie system powinien działać stabilnie przy obsłudze walk wieloobiektowych, stanu drużyny, ekwipunku, lochów proceduralnych oraz mechanizmu zapisu i odczytu danych.

Na obecnym etapie projektu można przyjąć orientacyjne wymagania minimalne:

- system operacyjny: Windows 10 lub nowszy,

- procesor: 64-bitowy, dwurdzeniowy,
- pamięć RAM: 8 GB,
- karta graficzna: zgodna z DirectX 11,
- wolne miejsce na dysku: co najmniej 2 GB.

Orientacyjne wymagania zalecane można określić następująco:

- system operacyjny: Windows 10 lub nowszy,
- procesor: 64-bitowy, czterordzeniowy,
- pamięć RAM: 16 GB,
- karta graficzna: dedykowana karta obsługująca DirectX 11 lub nowszy,
- wolne miejsce na dysku: 4 GB lub więcej.

Wymagania te mają charakter wstępny i mogą ulec zmianie w zależności od ostatecznego zakresu implementacji, liczby zasobów graficznych, rozbudowy GUI oraz stopnia złożoności finalnych mechanik. Na obecnym etapie stanowią one jednak wystarczającą podstawę do określenia docelowej platformy działania systemu.

2.3. Struktura systemu

Struktura systemu *Godmist* została zaprojektowana w sposób warstwowy i modularny. Jest to szczególnie istotne ze względu na złożoność projektowanej gry, która łączy wiele współdziałających podsystemów, takich jak eksploracja lochów, walka turowa, rozwój postaci, ekwipunek, crafting, interakcje z NPC, system misji oraz trwałość stanu gry. Przyjęta organizacja rozwiązania ma umożliwiać nie tylko poprawne działanie systemu, ale również jego późniejszą rozbudowę i utrzymanie.

Projekt nie opiera się na klasycznej architekturze bazodanowej charakterystycznej dla systemów biznesowych lub aplikacji sieciowych. Zamiast tego wykorzystuje strukturę danych dopasowaną do gry single player, w której istotną rolę odgrywają definicje statyczne, zmienny stan runtime oraz zapis trwały odtwarzany pomiędzy sesjami użytkownika. Oznacza to konieczność rozdzielenia tych trzech poziomów odpowiedzialności i opisania ich w sposób uporządkowany.

2.3.1. Opis modelu danych i trwałości

W projekcie *Godmist* przyjęto model danych oparty na rozdzieleniu definicji systemowych od danych runtime oraz trwałego zapisu stanu gry. Takie podejście pozwala uniknąć mieszania danych statycznych, które definiują reguły świata gry, z danymi zmiennymi, które opisują bieżący stan rozgrywki konkretnego użytkownika.

Pierwszą warstwę stanowią definicje statyczne. Są to dane opisujące elementy gry, które nie zmieniają się dynamicznie w ramach jednej kampanii, lecz pełnią rolę wzorców wykorzystywanych przez system. Do tej grupy należą między innymi definicje postaci, klas postaci, umiejętności, przedmiotów, NPC, questów i lochów. W architekturze projektu dane tego typu są reprezentowane jako obiekty `ScriptableObject`, co pozwala na ich czytelną organizację i łatwe rozwijanie w środowisku Unity.

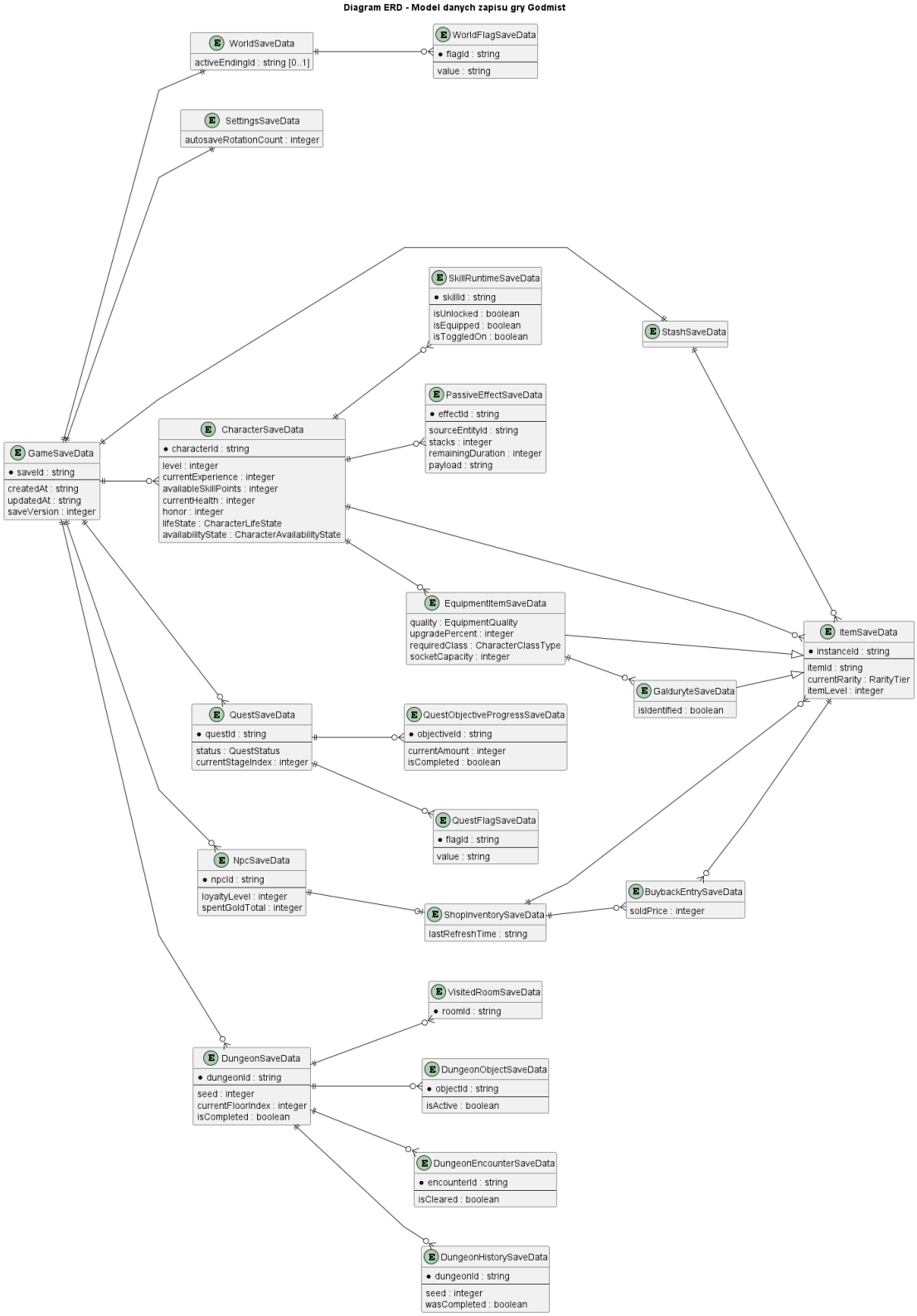
Drugą warstwę stanowią dane runtime, czyli bieżący stan gry podczas działania aplikacji. Warstwa ta obejmuje między innymi stan postaci, aktywne questy, stan lochu, ekwipunek, relacje między drużynami w walce, efekty pasywne oraz aktualne położenie i stan obiektów świata. Dane te są modyfikowane w trakcie rozgrywki i odzwierciedlają aktualną sytuację użytkownika.

Trzecią warstwę stanowi trwałość danych, czyli sposób zapisu i odtwarzania stanu gry pomiędzy sesjami. W projekcie nie zastosowano klasycznej relacyjnej bazy danych. Zamiast tego zapis realizowany jest z wykorzystaniem plików *JSON*. Rozwiązanie to jest uzasadnione charakterem gry single player, brakiem konieczności komunikacji z centralnym serwerem oraz potrzebą wygodnego odtwarzania rozbudowanego stanu gry. W zapisie przechowywane są między innymi dane postaci, stany przedmiotów, questów, NPC, lochów, schowka i świata.

Istotnym założeniem modelu danych jest identyfikowanie obiektów logicznych za pomocą stabilnych identyfikatorów tekstowych. Dzięki temu możliwe jest odwoływanie się do definicji danych oraz rekonstrukcja zależności pomiędzy obiektami podczas wczytywania gry. Przykładowo postać może odnosić się do definicji swojej klasy, przedmiot do definicji typu przedmiotu, a quest do definicji etapu lub celu. Takie podejście upraszcza serializację danych i wspiera modularny charakter projektu.

W przypadku lochów proceduralnych zastosowano podejście oparte na przechowywaniu ziarna generowania oraz zmian względem stanu bazowego. Oznacza to, że zamiast zapisywania pełnej, surowej kopii każdego elementu runtime, system może odtwarzać strukturę lochu na podstawie parametrów generacji i zapamiętanych różnic, takich jak odwiedzone pomieszczenia, pokonane walki czy aktywowane obiekty. Pozwala to ograniczyć złożoność zapisu i lepiej dopasować model trwałości do charakteru gry.

2.3.2. Diagram ERD



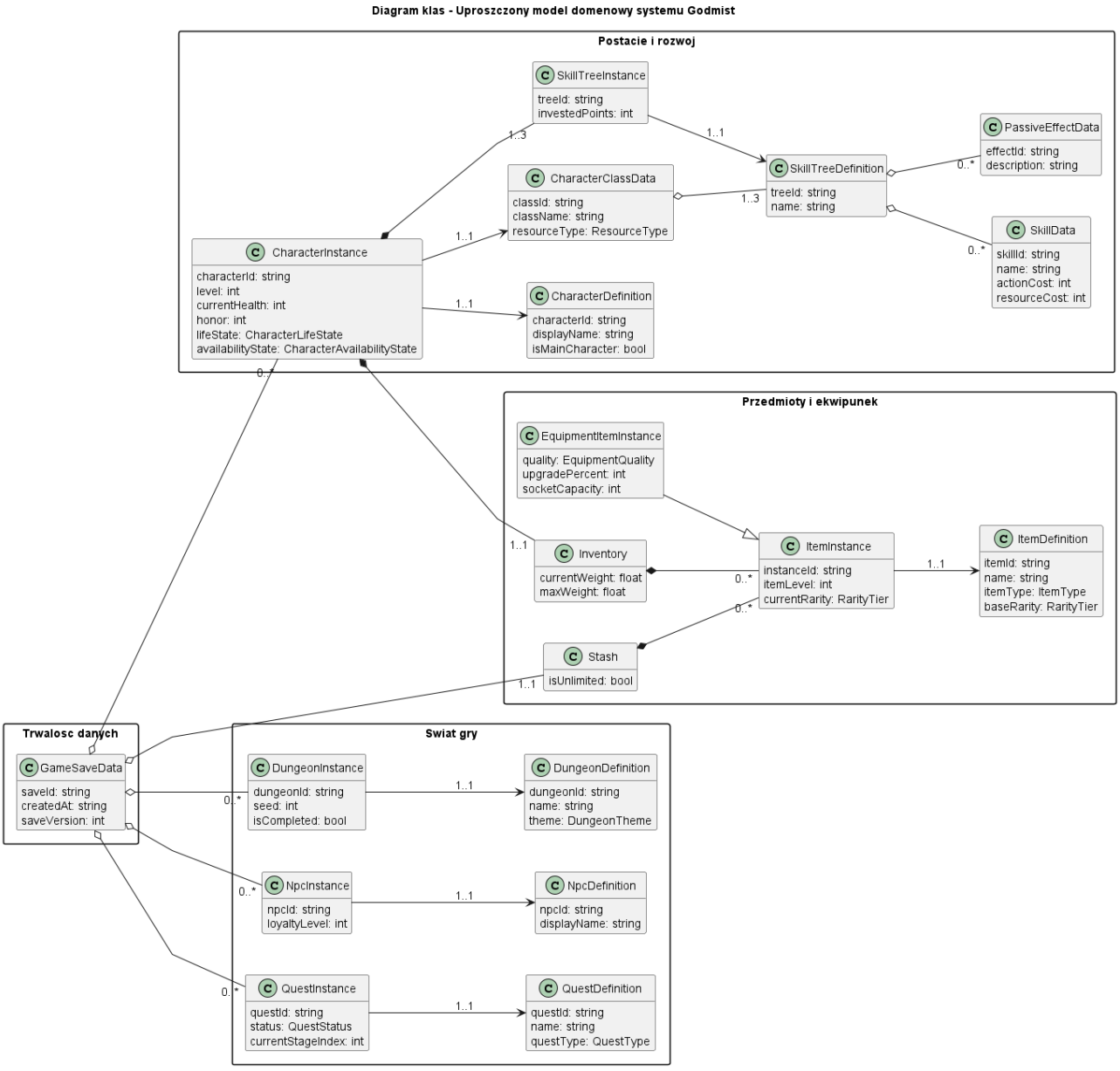
Rys. 2.10. Diagram ERD logicznego modelu danych zapisu gry Godmist.

Źródło: Opracowanie własne

Ze względu na charakter projektowanego systemu diagram ERD nie przedstawia klasycznej relacyjnej bazy danych, lecz logiczny model encji i zależności wykorzystywanych do zapisu stanu gry. Ujęto na nim najważniejsze struktury trwałe, takie jak zapis globalny, stan świata, postacie, questy, NPC, lochy

oraz dane związane z przedmiotami i schowkiem. Diagram pokazuje również relacje właścicielskie pomiędzy głównym obiektem zapisu a jego elementami składowymi, a także zależności pomiędzy specjalizacjami danych przedmiotów. Taka forma modelu danych lepiej odpowiada architekturze gry single player wykorzystującej pliki JSON niż klasyczny schemat tabel relacyjnych.

2.3.3. Diagram klas



Rys. 2.11. Uproszczony diagram klas systemu Godmist.

Źródło: Opracowanie własne

Na potrzeby niniejszej dokumentacji przedstawiono uproszczony diagram klas, którego celem jest pokazanie najważniejszych klas domenowych systemu oraz zależności pomiędzy nimi. Diagram nie stanowi pełnego odwzorowania wszystkich klas projektowych i implementacyjnych, lecz syntetyczny model obejmujący kluczowe obszary systemu: postacie i rozwój, przedmioty i ekwipunek, świat gry oraz trwałość danych. Takie uproszczenie zostało zastosowane celowo, aby zachować czytelność modelu i umożliwić jego analizę na poziomie architektury logicznej systemu.

Na diagramie widoczne są zależności pomiędzy definicjami danych, instancjami runtime oraz głównym obiektem zapisu gry. Pokazano między innymi relacje pomiędzy definicją postaci a jej instancją, powiązania klas postaci z drzewkami rozwoju, zależności pomiędzy instancjami przedmiotów i ich definicjami, a także relacje łączące zapis gry ze stanem postaci, questów, NPC, lochu i schowka. Dzięki temu diagram stanowi podsumowanie struktury domenowej systemu i uzupełnia wcześniejsze modele zachowania oraz model danych trwałych przedstawiony na diagramie ERD.

Warto podkreślić, że na diagramie pominięto część klas pomocniczych i szczegółowych mechanizmów, takich jak klasy widoku, serwisy systemowe, szczegółowe klasy walki, niższopoziomowe struktury zapisu oraz wybrane obiekty pomocnicze związane z mechanikami specjalistycznymi. Zabieg ten pozwolił skupić się na relacjach najistotniejszych z perspektywy projektowej i inżynierskiej. W efekcie diagram klas pełni w pracy rolę modelu poglądowego, porządkującego logikę architektury systemu, a nie pełnej dokumentacji implementacyjnej wszystkich elementów projektu.

2.3.4. Architektura warstwowa systemu

Projekt *Godmist* wykorzystuje architekturę warstwową, której celem jest rozdzielenie odpowiedzialności pomiędzy definicje danych, logikę domenową, serwisy systemowe oraz warstwę prezentacji. Takie podejście jest szczególnie korzystne w środowisku Unity, ponieważ pozwala ograniczyć nadmierne mieszanie logiki gry z klasami odpowiedzialnymi za scenę, widok i wejście użytkownika.

Pierwszą warstwę stanowią definicje danych. Obejmuje ona obiekty `ScriptableObject`, które przechowują statyczne informacje o klasach postaci, umiejętnościach, questach, przedmiotach, NPC i lochach. Warstwa ta pełni funkcję konfiguracji systemu i nie reprezentuje bezpośrednio zmiennego stanu rozgrywki.

Drugą warstwę stanowi model domenowy runtime. W jej skład wchodzą klasy reprezentujące aktualny stan postaci, walki, lochu, NPC, questów oraz innych elementów rozgrywki. To właśnie ta warstwa odpowiada za odwzorowanie bieżącej sytuacji w grze i przechowuje dane, które ulegają zmianom w trakcie działania aplikacji.

Trzecią warstwę tworzą serwisy systemowe. Ich zadaniem jest koordynowanie działania poszczególnych obszarów funkcjonalnych, takich jak walka, eksploracja, questy, inventory, crafting, NPC czy zapis gry. Serwisy te nie pełnią roli widoków ani definicji danych, lecz realizują logikę operacyjną i pośredniczą pomiędzy stanem gry a interakcjami użytkownika.

Czwartą warstwę stanowi warstwa prezentacji. Obejmuje ona klasy `MonoBehaviour`, które odpowiadają za wyświetlanie obiektów na scenie, obsługę interakcji użytkownika, integrację z UI oraz reagowanie na zmiany stanu systemu. W tej warstwie mieszczą się między innymi widoki postaci, NPC, obiektów lochu i okien interfejsu użytkownika.

Uzupełnieniem całej architektury jest warstwa trwałości danych, odpowiedzialna za serializację i deserializację stanu gry. Obejmuje ona klasy DTO wykorzystywane do zapisu w formacie JSON oraz komponenty odpowiedzialne za koordynację operacji zapisu i odczytu. Dzięki temu model definicji, runtime i trwałości pozostaje rozdzielony, co poprawia czytelność rozwiązania i ogranicza ryzyko niekontrolowanego przenikania się odpowiedzialności pomiędzy modułami.

Przyjęty podział warstwowy wspiera dalszy rozwój projektu, ponieważ umożliwia niezależną rozbudowę poszczególnych obszarów systemu. Pozwala również łatwiej identyfikować zależności pomiędzy elementami rozwiązania oraz zachować większą kontrolę nad architekturą w miarę wzrostu złożoności gry.

2.4. Podsumowanie rozdziału

W niniejszym rozdziale przedstawiono projekt systemu *Godmist* z perspektywy modelowania, doboru technologii oraz organizacji struktury rozwiązania. Omówiono rolę modeli UML jako narzędzia służącego do opisu zachowania i struktury systemu, wskazano główne technologie planowane do wykorzystania w projekcie oraz scharakteryzowano sposób organizacji danych i trwałości stanu gry.

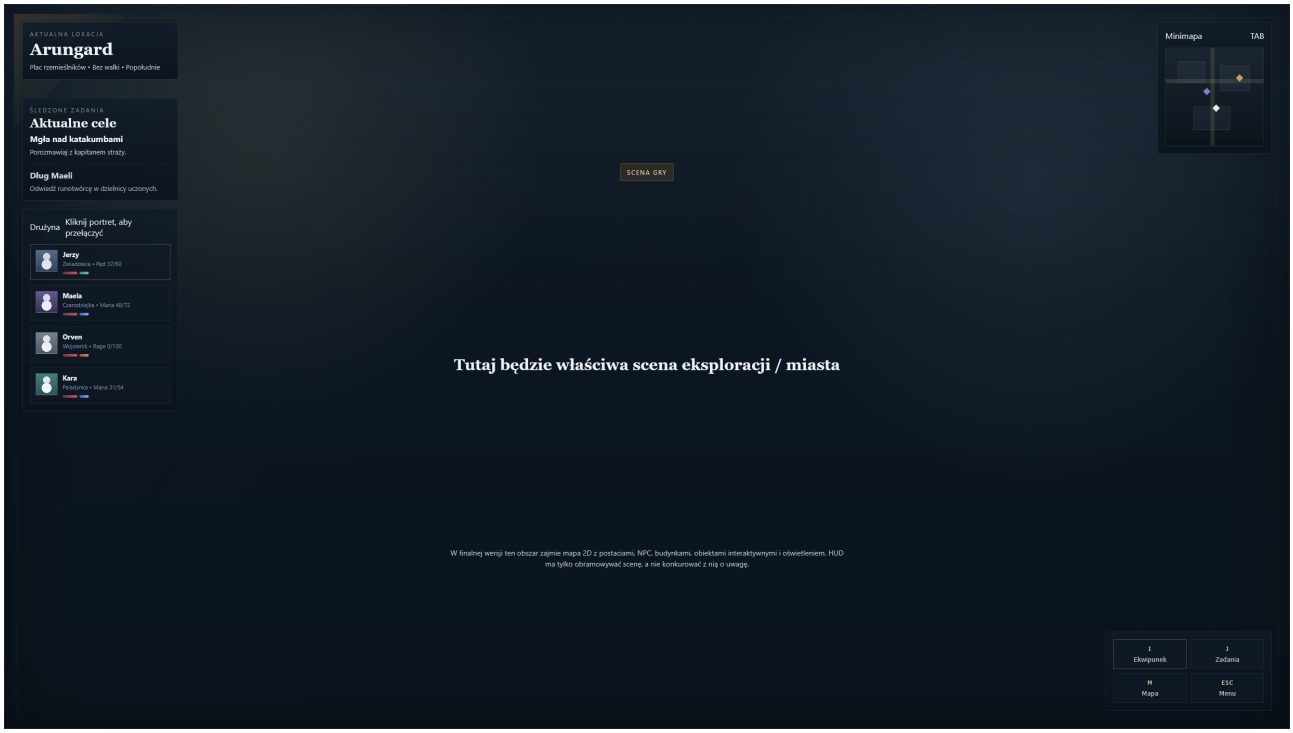
Szczególną uwagę zwrócono na fakt, że projekt nie wykorzystuje klasycznej relacyjnej bazy danych, lecz opiera się na rozdzieleniu definicji statycznych, stanu runtime oraz zapisu trwałego realizowanego z użyciem plików JSON. Pozwoliło to przedstawić architekturę rozwiązania w sposób lepiej dopasowany do charakteru gry single player oraz do przyjętych założeń technologicznych.

Przedstawiona struktura systemu stanowi podstawę do dalszego opisu bardziej praktycznych aspektów projektu. W kolejnym rozdziale omówiony zostanie interfejs użytkownika, propozycje testów oraz końcowe wnioski dotyczące projektowanego rozwiązania.

3. Interfejs użytkownika, testy i podsumowanie

3.1. Projekt interfejsu użytkownika

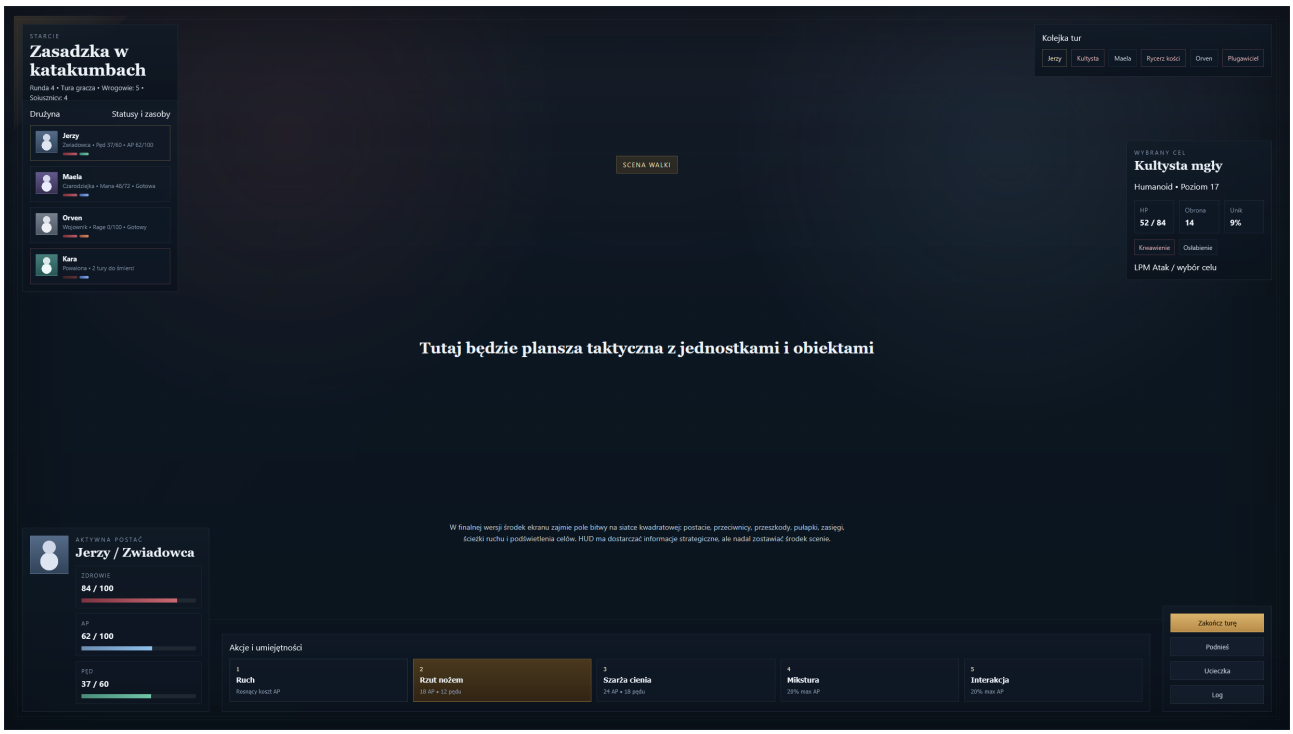
Interfejs użytkownika gry Godmist zaprojektowano zgodnie z zasadą minimalnej ingerencji w scenę gry. W widokach eksploracji oraz walki większość ekranu zajmuje właściwa scena, natomiast elementy HUD rozmieszczono przy krawędziach ekranu w formie lekkich widżetów. Bardziej rozbudowane okna, takie jak ekwipunek, dziennik zadań czy interakcje z NPC, otwierane są jako osobne nakładki.



Rys. 3.1. Widok eksploracji i miasta

Źródło: Opracowanie własne

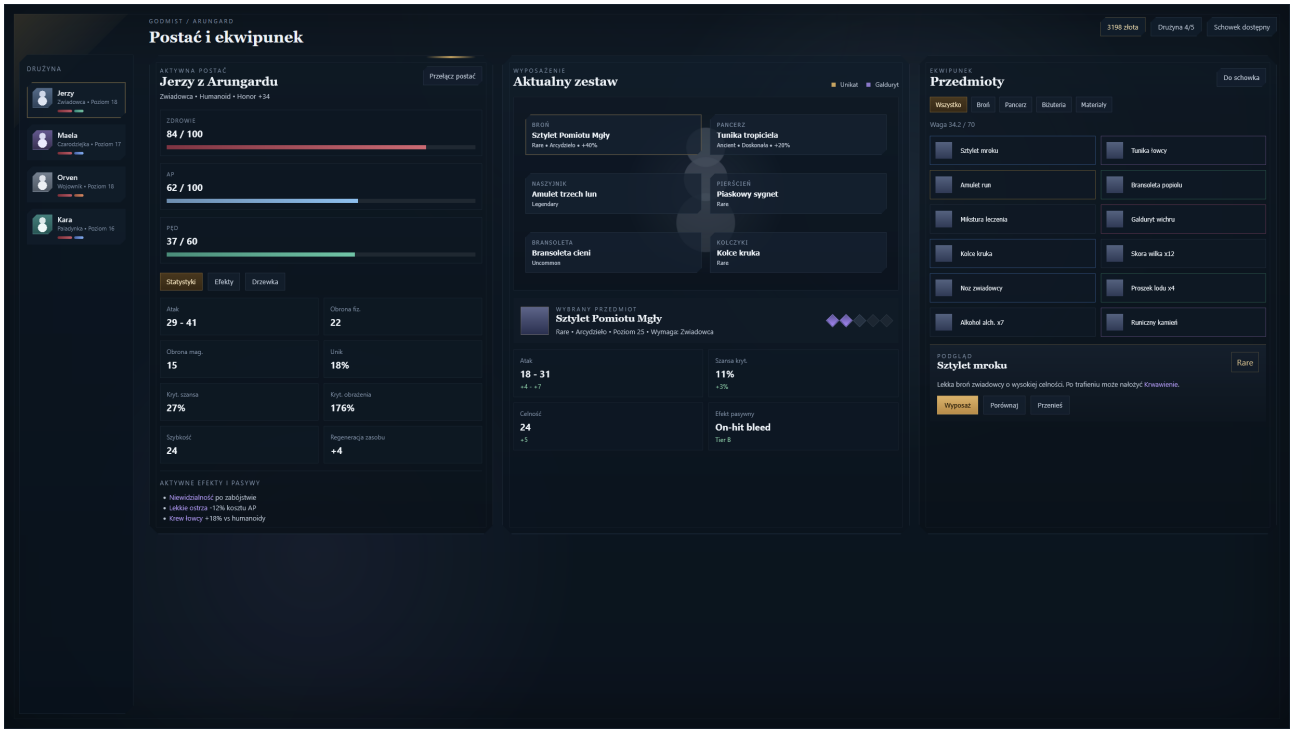
Widok eksploracji przedstawia podejście scene-first, w którym interfejs ograniczono do najważniejszych informacji: minimapy, aktywnych zadań, składu drużyny oraz podstawowych skrótów. Dzięki temu ekran nie zasłania przestrzeni gry i pozwala skupić uwagę użytkownika na otoczeniu.



Rys. 3.2. Widok interfejsu walki

Źródło: Opracowanie własne

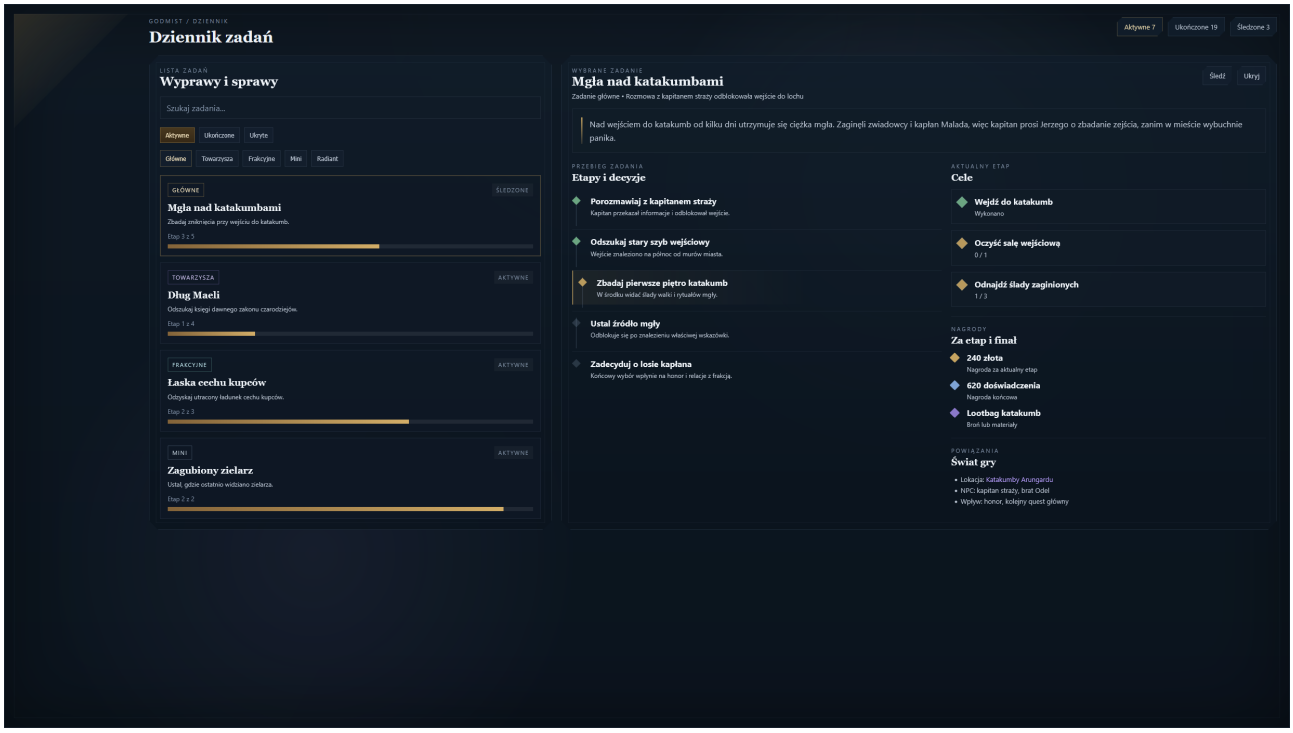
Interfejs walki zachowuje tę samą zasadę, jednak wzbogacono go o elementy strategiczne, takie jak kolejka tur, pasek umiejętności, panel aktywnej postaci oraz informacje o aktualnym celu. Rozmieszczenie tych elementów przy obrzeżach ekranu umożliwia jednoczesne śledzenie sytuacji taktycznej oraz samego pola bitwy.



Rys. 3.3. Widok postaci i ekwipunku

Źródło: Opracowanie własne

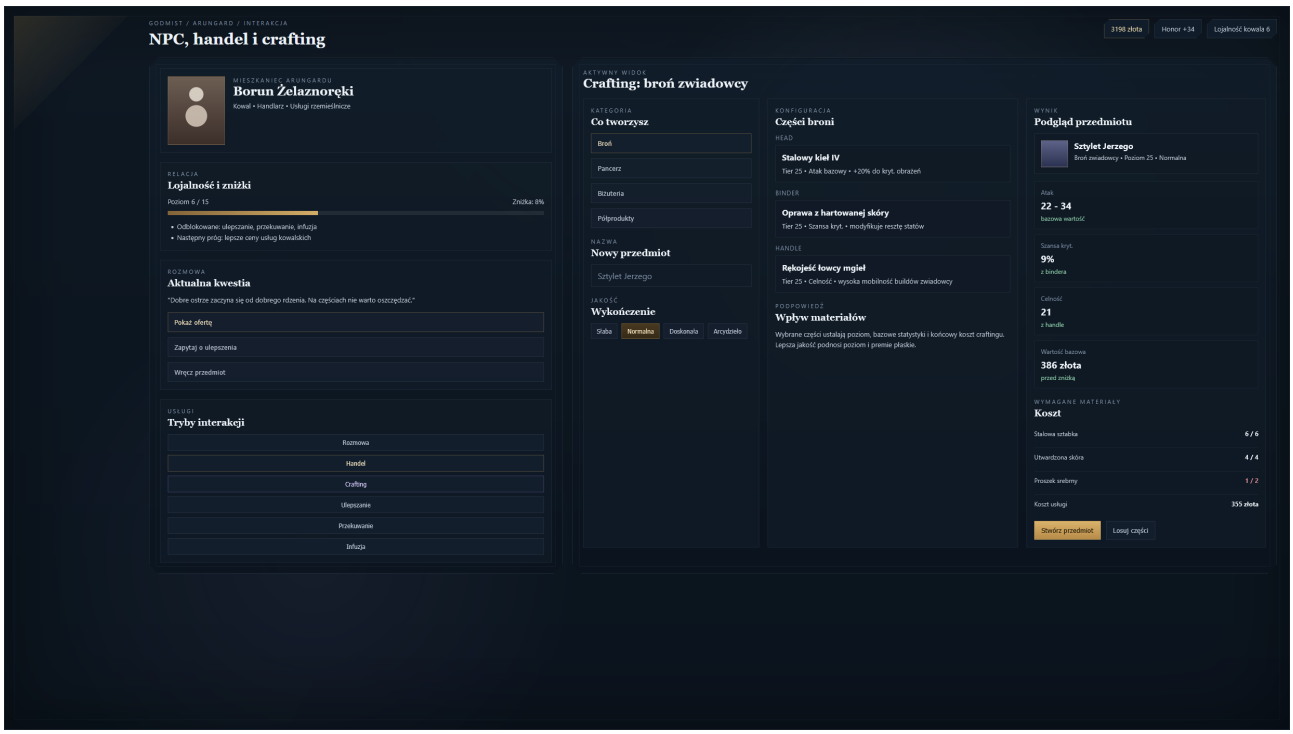
Okno postaci i ekwipunku umożliwia przegląd statystyk, aktualnie założonych przedmiotów oraz zawartości ekwipunku. Centralnym elementem kompozycji jest sekcja wyposażenia postaci, natomiast informacje pomocnicze rozmieszczono po bokach w celu zachowania czytelności.



Rys. 3.4. Widok dziennika zadań

Źródło: Opracowanie własne

Dziennik zadań grupuje informacje o aktywnych i ukończonych misjach, ich etapach, nagrodach oraz powiązaniach ze światem gry. Układ kolumnowy pozwala oddzielić listę zadań od szczegółów aktualnie wybranego wpisu.



Rys. 3.5. Widok interakcji z NPC

Źródło: Opracowanie własne

Okno interakcji z NPC stanowi wspólną bazę dla rozmowy, handlu oraz craftingu. Takie rozwiązanie upraszcza obsługę systemu i pozwala zachować spójność wizualną między różnymi usługami oferowanymi przez postacie niezależne.

3.2. Propozycja testów

Ze względu na złożoność systemu *Godmist* proces weryfikacji poprawności działania powinien obejmować kilka poziomów testowania. Oprócz sprawdzenia podstawowych funkcji dostępnych dla gracza konieczne jest również potwierdzenie poprawnej współpracy pomiędzy modułami systemu, prawidłowego działania zapisu i odczytu stanu gry oraz akceptowalnej wydajności rozwiązania. W niniejszej sekcji przedstawiono przykładowe scenariusze testowe, które mogą zostać wykorzystane podczas dalszej realizacji projektu.

Przedstawione przypadki testowe mają charakter projektowy. Oznacza to, że nie są jeszcze raportem z wykonanych testów, lecz propozycją zestawu sprawdzeń, które pozwolą zweryfikować zgodność implementacji z założeniami systemu. Dla czytelności testy podzielono na testy funkcjonalne, integracyjne, testy zapisu i odczytu oraz testy wydajnościowe i niezawodnościowe.

3.2.1. Testy funkcjonalne

Testy funkcjonalne mają na celu sprawdzenie, czy system poprawnie realizuje najważniejsze funkcje widoczne z perspektywy użytkownika końcowego. Obejmują one między innymi rozpoczęcie gry, eksplorację lokacji, prowadzenie walki, rozwój postaci, korzystanie z ekwipunku oraz obsługę handlu i craftingu.

Tab. 3.1. Przykładowe testy funkcjonalne systemu

ID	Nazwa testu	Warunki początkowe	Kroki testowe	Oczekiwany rezultat
TF-01	Rozpoczęcie nowej gry	Aplikacja została uruchomiona, użytkownik znajduje się w menu głównym.	Wybrać opcję rozpoczęcia nowej gry.	System tworzy nową kampanię i przenosi gracza do początkowego stanu rozgrywki.
TF-02	Eksploracja miasta	Gra została rozpoczęta, gracz znajduje się w mieście.	Poruszyć postać po mieście i wejść w interakcję z wybranym NPC.	Postać porusza się poprawnie, a interakcja z NPC otwiera odpowiedni panel lub dialog.
TF-03	Rozpoczęcie walki w lochu	Gracz znajduje się w lochu i zbliża się do grupy przeciwników.	Wejść w zasięg przeciwnika inicjujący walkę.	System przełącza stan gry do trybu walki i inicjuje sesję combatową.
TF-04	Użycie umiejętności w walce	Trwa walka, aktywna postać posiada wystarczające zasoby i punkty akcji.	Wybrać umiejętność i wskazać poprawny cel.	Umiejętność zostaje wykonana, a stan zasobów, punktów akcji i celu zostaje odpowiednio zaktualizowany.
TF-05	Handel z NPC	Gracz znajduje się w mieście i posiada przedmiot oraz odpowiednią ilość złota.	Otworzyć sklep NPC, kupić oraz sprzedać wybrany przedmiot.	System poprawnie aktualizuje złoto, inventory gracza oraz stan sklepu i listy buyback.
TF-06	Crafting przedmiotu	Gracz znajduje się przy odpowiednim NPC i posiada wymagane składniki.	Wybrać recepturę lub konfigurację przedmiotu i potwierdzić crafting.	System tworzy nowy przedmiot zgodnie z wybranymi parametrami i zużywa odpowiednie składniki.
TF-07	Rozwój postaci	Postać posiada dostępne punkty rozwoju.	Otworzyć drzewko umiejętności i zainwestować punkt w wybrany node.	System zwiększa inwestycję w node, aktualizuje liczbę punktów i odblokowuje odpowiednie efekty.

Źródło: Opracowanie własne

3.2.2. Testy integracyjne

Testy integracyjne służą do sprawdzenia poprawnej współpracy pomiędzy różnymi modułami systemu. W przypadku gry *Godmist* ma to szczególne znaczenie, ponieważ wiele mechanik korzysta z tych samych danych runtime i oddziałuje na stan świata, postaci oraz przedmiotów.

Tab. 3.2. Przykładowe testy integracyjne systemu

ID	Nazwa testu	Warunki początkowe	Kroki testowe	Oczekiwany rezultat
TI-01	Walka i efekty statusu	Trwa walka, aktywna postać posiada umiejętność nakładającą status.	Użyć umiejętności nakładającej efekt statusu na przeciwnika.	System poprawnie nakłada status, zapisuje go w stanie postaci i uwzględnia jego działanie w kolejnych turach.
TI-02	Ekwipunek i statystyki postaci	Postać posiada przedmiot zwiększający statystyki.	Założyć przedmiot na postać, a następnie go zdjąć.	Statystyki postaci zmieniają się po założeniu przedmiotu i wracają do poprzednich wartości po jego zdjęciu.
TI-03	Quest i stan świata	Aktywna jest misja wymagająca interakcji z obiektem lub NPC.	Wykonać czynność wymaganą przez quest.	System aktualizuje postęp questa oraz odpowiednie flagi świata lub questa.
TI-04	Loch i system walki	Gracz eksploruje loch zawierający encounter.	Wejść do pomieszczenia z przeciwnikami i ukończyć walkę.	System aktualizuje stan encountera w lochu i oznacza go jako ukończony.
TI-05	Crafting i inventory	Gracz posiada wymagane materiały do craftingu.	Wytworzyć przedmiot u odpowiedniego NPC.	Materiały zostają zużyte, a nowy przedmiot trafia do odpowiedniego inventory.

Źródło: Opracowanie własne

3.2.3. Testy zapisu i odczytu

Ze względu na przyjęty model trwałości danych oparty na plikach JSON, testy zapisu i odczytu należą do najważniejszych w całym projekcie. Ich celem jest weryfikacja, czy stan gry po zapisaniu i ponownym wczytaniu pozostaje zgodny ze stanem sprzed zakończenia sesji.

Tab. 3.3. Przykładowe testy zapisu i odczytu

ID	Nazwa testu	Warunki początkowe	Kroki testowe	Oczekiwany rezultat
TZ-01	Zapis i wczytanie postaci	Postać posiada określony poziom, statystyki, ekwipunek i aktywne efekty.	Zapisać grę, zamknąć aplikację, a następnie wczytać zapis.	Po wczytaniu system odtwarza stan postaci zgodny ze stanem sprzed zapisu.
TZ-02	Zapis i wczytanie lochu	Gracz znajduje się w lochu, odwiedził część pomieszczeń i ukończył wybrane encountery.	Zapisać grę i następnie wczytać zapis.	Stan lochu, w tym odwiedzone pomieszczenia i ukończone spotkania, zostaje odtworzony poprawnie.
TZ-03	Zapis i wczytanie questów	Gracz posiada aktywne questy z częściowym postępem.	Zapisać grę i ponownie ją wczytać.	System poprawnie odtwarza status questów, cele oraz postęp wykonania.
TZ-04	Zapis stanu NPC i handlu	NPC posiada określony poziom lojalności oraz stan sklepu i buybacku.	Zapisać grę po dokonaniu transakcji, a następnie ją wczytać.	Lojalność NPC, oferta sklepu i buyback pozostają zgodne z zapisanym stanem.
TZ-05	Zapis schowka	W schowku znajdują się różne przedmioty.	Odłożyć przedmioty do schowka, zapisać grę i ponownie ją wczytać.	Schowek po wczytaniu zawiera ten sam zestaw przedmiotów.

Źródło: Opracowanie własne

3.2.4. Testy wydajnościowe i niezawodnościowe

Testy wydajnościowe i niezawodnościowe mają na celu ocenę stabilności systemu oraz sprawdzenie, czy podstawowe operacje nie powodują zauważalnych opóźnień lub błędów przy większej złożoności stanu gry. W przypadku projektu *Godmist* istotne jest zwłaszcza zachowanie systemu przy rozbudowanym ekwipunku, wielu aktywnych efektach oraz przy obsłudze proceduralnie generowanych lochów.

Tab. 3.4. Przykładowe testy wydajnościowe i niezawodnościowe

ID	Nazwa testu	Warunki początkowe	Kroki testowe	Oczekiwany rezultat
TW-01	Ładowanie lochu proceduralnego	System jest gotowy do wejścia do lochu.	Wygenerować i załadować loch proceduralny.	Loch zostaje wygenerowany i załadowany bez błędów, a czas oczekiwania pozostaje akceptowalny dla użytkownika.
TW-02	Zapis dużego stanu gry	Gra zawiera rozwiniętą drużynę, dużą liczbę przedmiotów i aktywnych questów.	Wykonać zapis gry.	System zapisuje stan bez błędów i bez zauważalnego zawieszenia aplikacji.
TW-03	Wczytanie dużego stanu gry	Istnieje zapis obejmujący rozbudowany stan świata i postaci.	Wczytać zapis gry.	System poprawnie odtwarza stan gry bez utraty danych i bez krytycznych opóźnień.
TW-04	Stabilność długiej sesji gry	Użytkownik wykonuje wiele walk, interakcji i zapisów podczas jednej sesji.	Przeprowadzić dłuższą rozgrywkę obejmującą wiele przejść między modułami.	System zachowuje stabilność i nie wykazuje krytycznych błędów ani utraty spójności danych.

Źródło: Opracowanie własne

Przedstawiony zestaw testów nie wyczerpuje wszystkich możliwych scenariuszy weryfikacji systemu, ale stanowi reprezentatywną podstawę do dalszego planowania procesu testowania. Uwzględnia on zarówno działania wykonywane bezpośrednio przez użytkownika, jak i poprawność współpracy modułów, trwałość danych oraz stabilność techniczną systemu.

3.3. Podsumowanie rozdziału

W niniejszym rozdziale przedstawiono założenia dotyczące interfejsu użytkownika projektowanego systemu oraz zaproponowano przykładowe scenariusze testowe, które mogą zostać wykorzystane podczas dalszej realizacji projektu. Omówiono najważniejsze widoki GUI związane z menu głównym, miastem, eksploracją lochu, walką oraz zarządzaniem ekwipunkiem i rozwojem postaci. Pozwoliło to określić, w jaki sposób użytkownik będzie komunikował się z systemem w różnych kontekstach rozgrywki.

Dodatkowo zaprezentowano propozycję testów funkcjonalnych, integracyjnych, testów zapisu i odczytu oraz testów wydajnościowych i niezawodnościowych. Dzięki temu projekt został uzupełniony o perspektywę weryfikacji poprawności działania poszczególnych modułów. Przedstawione elementy stanowią praktyczne dopełnienie wcześniejszych rozdziałów poświęconych analizie, wymaganiom oraz modelowi systemu.

4. Podsumowanie

W niniejszej pracy przedstawiono projekt systemu gry komputerowej *Godmist* z perspektywy inżynierii oprogramowania. Opracowanie objęło analizę problemu, przegląd istniejących rozwiązań, określenie wymagań, modelowanie systemu, opis struktury danych, architektury rozwiązania, założeń interfejsu użytkownika oraz propozycji testów. Dzięki temu możliwe było przedstawienie projektu nie tylko jako koncepcji gry, ale również jako spójnego systemu informatycznego.

4.1. Ocena realizacji założeń

Założony cel pracy został osiągnięty w zakresie przygotowania dokumentacji projektowej systemu. Opracowano model rozwiązania obejmujący najważniejsze mechaniki gry, takie jak eksploracja lochów, walka turowa, rozwój postaci, system ekwipunku, crafting, obsługa NPC, questy oraz trwałość stanu gry. Przygotowane modele pozwalają uporządkować zależności pomiędzy poszczególnymi elementami systemu i stanowią podstawę do dalszych prac implementacyjnych.

Istotnym rezultatem pracy jest również rozdzielenie projektu na warstwy odpowiedzialności, obejmujące definicje danych, stan runtime, serwisy systemowe, warstwę prezentacji oraz trwałość danych. Takie podejście sprzyja modularności rozwiązania i zwiększa jego podatność na dalszą rozbudowę.

4.2. Wnioski końcowe

Przeprowadzona analiza pokazała, że projekt gry *Godmist* wymaga integracji wielu współdziałających podsystemów, które muszą pozostawać ze sobą w zgodzie zarówno na poziomie logiki rozgrywki, jak i modelu danych. Z punktu widzenia inżynierii oprogramowania najważniejsze okazało się odpowiednie rozdzielenie definicji danych od zmiennego stanu gry oraz zaprojektowanie spójnej struktury pośredniczącej pomiędzy eksploracją, walką, przedmiotami, misjami i zapisem gry.

Dokumentacja przygotowana w ramach pracy porządkuje te zależności i tworzy podstawę do dalszej realizacji projektu. Można zatem uznać, że *Godmist* został poprawnie opisany jako projekt systemu informatycznego o złożonej strukturze i wielu współzależnych modułach.

4.3. Możliwe kierunki dalszego rozwoju

Projekt może być w przyszłości rozwijany zarówno od strony technicznej, jak i projektowej. Do najważniejszych kierunków dalszych prac należą:

- pełna implementacja systemu w środowisku Unity,
- rozbudowa dialogów i warstwy fabularnej,
- rozwinięcie systemu lochów, przeciwników i zagadek,
- dalsze dopracowanie interfejsu użytkownika,
- rozszerzenie zestawu testów o testy wykonywane na działającej implementacji,
- balansowanie mechanik związanych z walką, rozwojem postaci i przedmiotami.

Przedstawiona dokumentacja może stanowić punkt wyjścia do dalszej realizacji projektu oraz kolejnych iteracji rozwojowych systemu.

Spis rysunków

2.1 Diagram przypadków użycia dla sesji gry, eksploracji i walki. 12
2.2 Diagram przypadków użycia dla zarządzania postacią i ekwipunkiem. 13
2.3 Diagram przypadków użycia dla handlu, craftingu i questów. 14
2.4 Diagram aktywności przedstawiający proces craftingu u NPC. 18
2.5 Diagram aktywności przedstawiający przyjęcie i realizację questa. 19
2.6 Diagram sekwencji przedstawiający handel z NPC. 20
2.7 Diagram sekwencji przedstawiający przełączenie aktywnej postaci poza walką. . . . 21
2.8 Diagram stanów obiektu CharacterLifeState. 22
2.9 Diagram stanów obiektu QuestInstance. 23
2.10 Diagram ERD logicznego modelu danych zapisu gry Godmist. 27
2.11 Uproszczony diagram klas systemu Godmist. 28

3.1 Widok eksploracji i miasta 31
3.2 Widok interfejsu walki 32
3.3 Widok postaci i ekwipunku 32
3.4 Widok dziennika zadań 33
3.5 Widok interakcji z NPC 34

Spis tabel

3.1 Przykładowe testy funkcjonalne systemu 35
3.2 Przykładowe testy integracyjne systemu 36
3.3 Przykładowe testy zapisu i odczytu 37
3.4 Przykładowe testy wydajnościowe i niezawodnościowe 38

Spis listingów

Bibliografia

[1] Ernest Adams. *Fundamentals of Game Design*. New Riders, 3 edition, 2014.

[2] Tracy Fullerton. *Game Design Workshop: A Playcentric Approach to Creating Innovative Games*. CRC Press, 4 edition, 2018.

[3] Larian Studios. Baldur's gate 3 – official website. `https://baldursgate3.game/`, 2026. Dostęp: 29.06.2026.

[4] Microsoft. *C# Documentation*. Microsoft, 2026. Dostęp: 29.06.2026.

[5] Microsoft. *How to Serialize and Deserialize JSON in .NET*. Microsoft, 2026. Dostęp: 29.06.2026.

[6] Robert Nystrom. *Game Programming Patterns*. Genever Benning, 2014.

[7] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. McGraw-Hill, 9 edition, 2020.

[8] Red Hook Studios. Darkest dungeon – official website. `https://www.darkestdungeon.com/`, 2026. Dostęp: 29.06.2026.

[9] Scott Rogers. *Level Up! The Guide to Great Video Game Design*. Wiley, 2 edition, 2014.

[10] Sandfall Interactive. Clair obscur: Expedition 33 – official website. `https://www.expedition33.com/`, 2026. Dostęp: 29.06.2026.

[11] Jesse Schell. *The Art of Game Design: A Book of Lenses*. CRC Press, 3 edition, 2019.

[12] Ian Sommerville. *Software Engineering*. Pearson, 10 edition, 2015.

[13] Unity Technologies. *JSON Serialization – Unity Manual*. Unity Technologies, 2026. Dostęp: 29.06.2026.

[14] Unity Technologies. *MonoBehaviour – Unity Scripting API*. Unity Technologies, 2026. Dostęp: 29.06.2026.

[15] Unity Technologies. *ScriptableObject – Unity Manual*. Unity Technologies, 2026. Dostęp: 29.06.2026.

[16] Unity Technologies. *ScriptableObject – Unity Scripting API*. Unity Technologies, 2026. Dostęp: 29.06.2026.

[17] Unity Technologies. *Unity Manual*. Unity Technologies, 2026. Dostęp: 29.06.2026.

[18] Valve Corporation. Baldur's gate 3 on steam. `https://store.steampowered.com/app/1086940/Baldurs_Gate_3/`, 2026. Dostęp: 29.06.2026.

[19] Valve Corporation. Clair obscur: Expedition 33 on steam. https://store.steampowered.com/app/1903340/Clair_Obscur_Expedition_33/, 2026. Dostęp: 29.06.2026.

[20] Valve Corporation. Darkest dungeon on steam. https://store.steampowered.com/app/262060/Darkest_Dungeon/, 2026. Dostęp: 29.06.2026.